МИНИСТЕРСТВО ОБРАЗОВАНИЯ ОРЕНБУРГСКОЙ ОБЛАСТИ
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ СРЕДНЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«ОРЕНБУРГСКИЙ КОЛЛЕДЖ ЭКОНОМИКИ И ИНФОРМАТИКИ»
(ГАПОУ СПО ОКЭИ)

КУРСОВОЙ ПРОЕКТ

ОКЭИ 09.02.07. 4026. 13 О
(код документа)

Разработка веб-приложения «RPG Nexus» — чата с игровыми персонажами

(тема проекта)

Количество листов 41

Дата готовности 09.11.2026

Обучающийся Плейман Лев Сергеевич, 3ВБ2

Руководитель Коноводов Н.С.

Защищен_____ с оценкой _____

Оренбург 2026

Содержание

Введение 3
1 Техническое задание 5
2 Разработка проектной документации 8
2.1 Подбор графических материалов 8
2.2 Разработка макета и прототипа веб-приложения 10
3 Разработка веб-приложения 13
3.1 Создание шаблонов веб-страниц 13
3.2 Реализация динамических элементов на сайте 14
3.3 Реализация кода сервера 17
3.4 Реализация базы данных 20
4 Тестирование веб-приложения 24
Заключение 26
Список используемых источников 28
Приложение А *(обязательное)* Входные документы 29
Приложение Б *(обязательное)* Выходные документы 30
Приложение В *(обязательное)* Информационная модель 31
Приложение Г *(обязательное)* Функциональная модель 32
Приложение Д *(обязательное)* Диаграмма прецедентов 33
Приложение Е *(обязательное)* Дизайн сайта 34
Приложение Ж *(обязательное)* Листинг программы 39

Введение

В настоящее время информационные технологии активно используются в сфере цифрового досуга, интерактивной литературы и компьютерных игр. Отдельное направление занимают текстовые ролевые приключения, в которых игрок влияет на развитие сюжета посредством собственных решений. В отличие от традиционного чтения пользователь получает возможность выбирать реплики, вступать во взаимодействие с героями, собирать предметы и наблюдать последствия своих поступков. Поэтому веб-приложение для текстовых RPG должно объединять функции диалога, управления состоянием истории и представления игровых сведений в едином понятном интерфейсе.

Одним из востребованных форматов такого взаимодействия становится чат с виртуальным персонажем. Пользователь выбирает собеседника, знакомится с его характером и биографией, а затем участвует в разворачивающемся приключении. Этот формат удобен тем, что не требует специального игрового оборудования: достаточно современного браузера. При этом возникает задача грамотно организовать логику ответов и сценариев, чтобы выбор пользователя действительно изменял сюжет, а не ограничивался повторением одинаковых сообщений.

Актуальность темы заключается в сочетании нескольких направлений разработки: проектирования удобных интерфейсов, работы с данными, программирования событийных систем и моделирования игровых правил. Для пользователя важны выразительный визуальный стиль, быстрый поиск персонажа и возможность вернуться к прерванной истории. Для разработчика необходимо обеспечить модульность исходного кода, понятный формат сценариев, проверку пользовательского ввода и предсказуемое сохранение. Отдельное внимание должно уделяться адаптивному представлению чата, так как на небольшом экране одновременно разместить переписку, список сеансов и показатели героя сложнее, чем на компьютере.

В рамках работы рассматривается веб-приложение «RPG Nexus» – русскоязычный прототип для текстовых RPG-приключений. В архиве проекта содержатся десять самостоятельных HTML-страниц, двенадцать готовых персонажей и шесть игровых миров. Приложение позволяет открывать карточки героев, выбирать тематический мир, писать сообщения, пользоваться предложенными действиями, применять двадцатигранный кубик D20, получать опыт и игровые предметы. Предусмотрены собственные персонажи, локальная история прохождений, редактирование профиля и переключение темы оформления.

Цель работы заключается в проектировании и документировании расширенной версии веб-приложения «RPG Nexus» – чата с игровыми ботами и системой текстовых ролевых приключений. Помимо существующих игровых возможностей, техническое задание предусматривает создание серверной части, реляционной базы данных, регистрации пользователей и безопасного сохранения прогресса. Для достижения цели необходимо определить требования, описать

клиентские и серверные компоненты, разработать информационную модель и спланировать проверку системы.

Объектом исследования является процесс взаимодействия игрока с виртуальными персонажами в текстовой ролевой игре. Предмет исследования – методы разработки многостраничного веб-приложения, серверной обработки игровых команд, реляционного хранения сообщений, персонажей и игровых сессий. Практическая значимость состоит в подготовке согласованной архитектуры, по которой локальный прототип может быть расширен до многопользовательского сервиса с разграничением доступа.

В исходном архиве использованы «HTML5», «CSS3», «JavaScript» и модульная структура клиентского кода. В новой архитектуре по техническому заданию клиент обменивается данными с сервером «Node.js 20+» и «Express» посредством «REST API», а сведения сохраняются в «PostgreSQL 16» через библиотеку «pg». Регистрация и вход проектируются с использованием «bcrypt» и токенов «JWT». «Docker Compose» должен запускать отдельные контейнеры приложения и базы данных. Файлы «JSON» остаются источником первоначального наполнения, а не основным пользовательским хранилищем.

Важным свойством продукта является связь между разговором и механикой RPG. Система не только сохраняет сообщения, но и рассчитывает последствия действий: уменьшает или восстанавливает здоровье, изменяет запас маны, начисляет опыт и золото, добавляет предметы в инвентарь. Некоторые решения требуют успешного броска D20, что приводит к различным сюжетным веткам. Созданный пользователем персонаж также может участвовать в универсальной истории. Такой подход показывает, как интерфейс текстового чата превращается в оболочку для самостоятельной игровой системы.

Структура отчета отражает основные этапы работы. В первом разделе сформулированы функциональные, нефункциональные и технические требования. Во втором приведены материалы проектирования и макеты. В третьем рассмотрены шаблоны страниц, взаимодействие интерфейса с программными маршрутами, серверные правила и модель из десяти таблиц. В четвертом определены проведенные проверки исходного прототипа и обязательные испытания новой серверной версии. Приложения содержат схемы, диаграмму прецедентов, входные и выходные данные, изображения и проектные фрагменты программного кода.

При выборе архитектуры учитывается дальнейшее развитие проекта: каталог персонажей и миров должен допускать добавление новых материалов, а личные данные и история каждого игрока должны храниться отдельно от общедоступного содержимого.

В отчете разграничиваются сведения об исходном архиве и требования к расширенной версии, чтобы проектируемые серверные функции не представлялись как уже проверенные.

1 Техническое задание

Наименование продукта – «RPG Nexus: чат с игровыми ботами». Цель доработки – переход от автономного браузерного приложения к клиент-серверной системе, в которой регистрация, персонажи, избранное, профиль, сообщения, инвентарь и игровые сессии сохраняются в централизованной реляционной базе данных. При этом необходимо сохранить существующие десять страниц, внешний вид, адаптивность, шесть миров, двенадцать официальных героев и игровую механику с броском D20.

Основанием для проектирования служит техническое задание на разработку серверной части и базы данных для «RPG Nexus». Требуется использовать «Node.js» версии 20 и выше, «Express», библиотеку «pg», «bcrypt», «jsonwebtoken» и «PostgreSQL 16». Клиентская часть остается на «HTML», «CSS» и обычном «JavaScript» без интерфейсных фреймворков. Сетевая схема: браузер – защищенный «REST API» – сервер приложений – база данных; прямые подключения браузера к базе запрещены.

Основной целевой группой являются пользователи, увлекающиеся ролевыми играми, интерактивными историями и фантастическими мирами. Им требуется простой вход в игру без установки отдельного клиента, удобное управление перепиской и наглядное представление последствий своих решений. Второй группой являются создатели собственных героев, заинтересованные в настройке биографии, характера, приветствия и стартового сюжета. Третья группа – разработчики и проверяющие, которым необходим воспроизводимый запуск, понятные файлы проекта и возможность выполнить автоматические тесты.

Предусмотрены десять существующих страниц: главная, каталог, карточка персонажа, игровой чат, конструктор героя, собственная коллекция, игровые миры, история приключений, профиль и авторизация. Они сохраняют прежнюю разметку и стили, но начинают получать изменяемые данные по «API». Дополнительно допускается создание страницы администратора для управления учетными записями, официальными персонажами, мирами и сценариями.

Главная страница должна показывать название платформы, заметную кнопку начала приключения, переход к игровым мирам, подборку персонажей и преимущества приложения. При наличии сохраненной истории следует предоставлять быстрый возврат к недавним сессиям. В каталоге необходимо реализовать поиск, жанровые фильтры, сортировку и вывод избранного. Карточка персонажа должна содержать его имя, жанр, роль, биографию, черты характера, способности, описание и ссылку на запуск игры.

Страница чата является основным рабочим экраном. Она должна содержать историю переписки, поле ввода, доступные сюжетные действия, сведения о собеседнике и характеристики игрока. На настольном компьютере допускается трехколоночное размещение с историей сеансов слева, диалогом в центре и панелью героя справа. На мобильных устройствах второстепенные области следует скрывать за переключаемыми вкладками, оставляя диалог главным элементом интерфейса.

Каждая игровая сессия должна иметь уникальный идентификатор «UUID», идентификаторы владельца, персонажа и мира, текущий сюжетный узел, состояние active/completed/failed, сообщения, время создания и изменения и характеристики героя. Для сохранения истории используются снимки имени и аватара персонажа, названия мира. Это позволяет показывать прежние приключения после мягкого удаления пользовательского героя. Другие пользователи не вправе читать и менять чужие сессии.

Начальные характеристики новой сессии: уровень 1, здоровье 100, мана 50, опыт 0, золото 50. Начальный инвентарь содержит «Походный рюкзак» и «Зелье здоровья». Каждые 100 единиц опыта повышают уровень на единицу с восстановлением до 20 здоровья и до 10 маны в пределах максимума. Команда употребления зелья восстанавливает до 35 здоровья и расходует один предмет. Бросок D20 выполняется на сервере и возвращает целое число от 1 до 20.

Ввод сообщения должен поддерживать две модели взаимодействия. Первая – выбор заранее предусмотренного действия через кнопку или номер варианта. Вторая – текстовое сообщение, по которому система ищет соответствующие ключевые слова. При отсутствии совпадения бот должен выдавать безопасный нейтральный ответ и повторно предлагать доступные действия. Это поведение важно, поскольку в первой версии не используется свободно генерирующая ответы нейросеть.

Конструктор персонажей должен предоставлять форму с именем, ролью, жанром, описанием, биографией, особенностями поведения, способностями, приветствием, начальной ситуацией и аватаром. Пользователь должен иметь возможность увидеть предварительный результат, сохранить персонажа, открыть его для редактирования либо удалить после подтверждения. Созданные герои без связанного специального мира должны участвовать в универсальной ветвящейся истории, использующей введенное описание.

Основным хранилищем новых пользовательских данных должна стать база «PostgreSQL 16». Модель включает десять таблиц: users, profiles, worlds, characters, favorites, scenario_nodes, scenario_actions, game_sessions, messages и session_inventory. Исходные файлы data/bots.json и data/worlds.json используются только при начальном заполнении базы. В браузере допустимо сохранять токен «JWT» и необязательный кэш темы оформления; профиль, историю, персонажей и избранное хранить в «localStorage» после миграции нельзя.

Безопасность предусматривает хеширование паролей библиотекой «bcrypt» с десятью раундами, аутентификацию по «JWT» из заголовка Authorization, проверку роли и владельца объекта, параметризованные запросы, транзакции, ограничение размера запроса и серверную проверку полей. Администратор управляет официальными персонажами, мирами и учетными записями; игрок изменяет только свои данные. Пользовательский текст отображается через textContent. Сервер не должен принимать от клиента произвольные значения здоровья, опыта, золота, инвентаря или результаты D20.

Нефункциональные требования включают корректное отображение десяти страниц на экранах 320, 768 и 1440 пикселей, работу без горизонтальной прокрутки, понятную обработку ошибок и отсутствие потери данных при

перезагрузке страницы. Сервер должен отдавать статические ресурсы, предоставлять маршруты «/api», возвращать ошибки в формате JSON с кодами 400, 401, 403, 404, 409 и 500, а также запускаться локально и посредством «Docker Compose».

Серверная структура по заданию выделяет каталог public с прежними HTML-страницами и ресурсами, каталог server с маршрутизаторами, подключением к базе, служебным заполнением и игровой логикой, а также папку tests. Основные файлы: server/index.js, server/db.js, server/seed.js, server/middleware/auth.js, server/routes и server/services/rpg-engine.js. Для запуска предусмотрены Dockerfile, docker-compose.yml, .env.example, package.json, start.bat и README.md.

Критерии готовности серверной версии включают наличие десяти связанных таблиц, импорт двенадцати официальных персонажей, шести публичных миров, сорока двух сценарных узлов и девяноста сюжетных действий. Кроме того, должны работать создание учетной записи, вход, управление профилем и избранным, создание героев, запуск и продолжение приключений, история и роли администратора. Необходимы автоматические испытания «API», проверка запрета чужих действий и защита от подмены игровых характеристик.

Сервис должен запускаться с двумя контейнерами: app и database. База использует образ postgres:16-alpine и постоянный том, а приложение ожидает успешной проверки готовности базы. Основная команда – docker compose up --build; для запуска с базой в контейнере и программой на компьютере предусмотрены npm install, npm run seed и npm start. На «Windows» запуск упрощается файлом start.bat. Секрет токенов и пароль базы передаются в переменных окружения, а не записываются в клиентские файлы.

Техническое задание требует разделить публичные и закрытые операции. Гость читает каталог героев и игровые миры. Зарегистрированный игрок получает личные функции и доступ только к собственным сессиям. Администратор может управлять учетными записями и содержимым игровых миров. При удалении пользовательского персонажа применяется мягкое удаление с сохранением истории; удаление сессии приводит к согласованному удалению ее сообщений и предметов.

Функциональные запросы сгруппированы по разделам «/api/auth», «/api/profile», «/api/characters», «/api/favorites», «/api/worlds», «/api/sessions» и административным маршрутам. Игровая логика переносится на сервер, а каждый игровой ход сохраняется в одной транзакции «PostgreSQL» с блокировкой строки сессии через SELECT FOR UPDATE. В данном отчете новые серверные модули и базы описаны по переданному ТЗ; фактическое выполнение их испытаний необходимо подтвердить после получения готовой обновленной сборки.

Приемка проектируемой системы включает проверку регистрации и входа, получения каталога, создания персонажа, запуска и продолжения приключения, отправки сообщений и сохранения состояния после повторного входа. Дополнительно проверяется защита личных сессий и невозможность подмены игровых показателей.

2 Разработка проектной документации

2.1 Подбор графических материалов

Тема текстовых RPG предполагает атмосферное оформление. Важную роль играют иллюстрации локаций, узнаваемые портреты героев и визуальные акценты, которые помогают погрузиться в историю. При этом основной контент приложения – текст переписки, поэтому фон и декоративные элементы не должны снижать его читаемость. Для «RPG Nexus» выбрана темная палитра с контрастными светлыми надписями, золотистыми кнопками и более спокойными поверхностями карточек и боковых панелей.

Графические материалы в исходном архиве организованы по назначению. В каталоге assets/images/bots размещены изображения персонажей, в assets/images/worlds – фоны игровых миров, в assets/images/avatars – аватары профиля, в assets/images/backgrounds – крупная композиция главного экрана. Значок приложения находится в assets/icons. Такое распределение позволяет быстро определить назначение каждого ресурса, а также не смешивать иллюстрации с текстовыми данными и исполняемыми модулями.

В проекте используются локальные SVG-изображения. Данный формат подходит для тематических иллюстраций, поскольку масштабирование сохраняет четкость геометрических линий. Векторные материалы обычно хорошо работают на экранах с различной плотностью пикселей. Вместе с тем сложные изображения содержат множество элементов и требуют контроля размера файлов. Поэтому при подготовке ресурсов необходимо оценивать не только эстетические свойства, но и объем загружаемых данных.

Для персональных карточек героев изображения особенно важны: они формируют первое впечатление о характере персонажа и жанре его истории. Лесная хранительница, воин, кибернетический персонаж и маг должны легко различаться еще до чтения подробного описания. На странице одного бота портрет занимает больше пространства и сопровождается биографией, способностями и чертами характера. В каталоге изображение рассматривается прежде всего как элемент быстрого ориентирования.

Иллюстрации миров должны передавать особенности сценария без избыточного текста. Фантастическое королевство, футуристический город или космическая среда предполагают различные образы и палитры. Для игровых миров в JSON-файле указывается путь к соответствующему изображению. Это решение отделяет визуальный материал от структуры карточки: новый мир может использовать общий шаблон, меняя название, описание и иллюстрацию.

Первый экран проектируется как композиция из центрального заголовка, фоновой тематической сцены и двух важных действий: выбор персонажа и переход к мирам. Чрезмерное количество равнозначных кнопок на первом экране могло бы затруднить начало игры, поэтому второстепенные возможности переносятся в навигационное меню и блоки ниже. При наличии старых сессий появляется отдельная область быстрого продолжения истории.

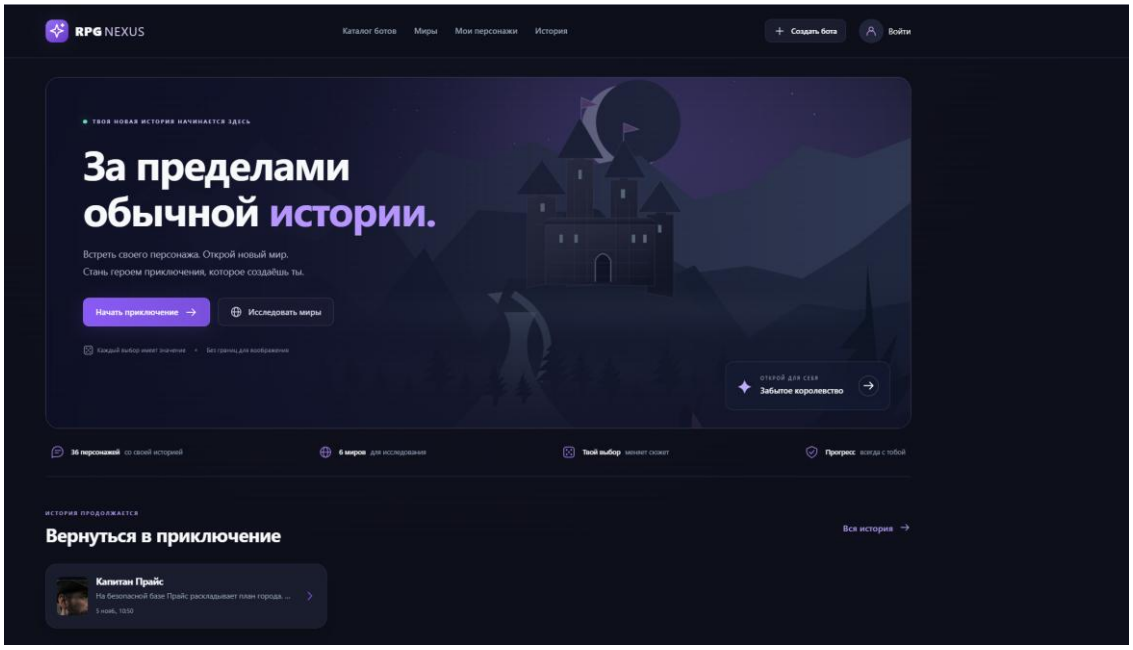


Рисунок 1 – Эскиз главной страницы с использованием материалов проекта

На рисунке представлен именно проектный эскиз, созданный по структуре HTML-страницы и с применением SVG-материалов исходного архива. Он позволяет рассмотреть расположение главного призыва к действию, навигации и сетки персонажей. Изображение не является скриншотом фактического исполнения в браузере и используется для иллюстрации принципов композиции и выбора графических материалов.

В оформлении принята многоуровневая система контраста. Общий фон должен оставаться достаточно темным для создания игрового настроения. Контейнеры сообщений и карточек выделяются более светлым оттенком. Главная кнопка отличается цветом и плотностью заливки. Служебные подписи, отметки времени и второстепенные ссылки обладают меньшей визуальной насыщенностью. Такое разделение помогает игроку концентрировать внимание на переписке и текущем выборе.

В качестве основного графического материала для интерфейса чата рекомендуется использовать не фоновые изображения, а текст, иконки и стандартные элементы управления. В отличие от главной страницы чат может оставаться открытым долго, поэтому чрезмерно активный фон утомляет и мешает воспринимать большие сообщения. Боковые панели допускают небольшие портреты, показатели и индикаторы, но центральная область должна сохранять достаточную свободную площадь.

Доступность графики является самостоятельной задачей. Каждое значимое изображение требует альтернативного описания, а действие не должно определяться исключительно цветом. Например, состояние добавления персонажа в избранное можно отражать одновременно цветом кнопки, подписью и aria-pressed. При неудачной загрузке изображения в приложении предусмотрен запасной локальный ресурс. Эти меры помогают сохранить работоспособность экрана при частичной недоступности материалов.

README в папке assets указывает, что SVG-материалы разработаны специально для данного проекта. Поэтому в отчете они описываются как ресурсы исходного архива. Справочные внешние сайты, включенные в список использованных источников, могут применяться при изучении адаптивности и принципов веб-дизайна, но их упоминание не означает копирования чужих изображений в программу.

2.2 Разработка макета и прототипа веб-приложения

Многостраничная структура из десяти экранов сохраняется без изменения принципа навигации. После выбора персонажа или мира клиент обращается к «API» и получает публичные сведения. Для запуска игры требуется вход: сервер создает сессию для текущего пользователя, а страница чата отображает выданные сообщения, действия, характеристики и инвентарь. Переписка и состояние загружаются повторно из базы при возвращении к прежнему приключению.

На главной странице располагаются название проекта, вводный слоган, основные кнопки, список популярных собеседников, подборка миров и объяснение правил. Из меню пользователь переходит к каталогу, истории, коллекции или профилю. Каталог содержит поиск и фильтры, а карточка персонажа раскрывает подробности, которые помогают решить, начинать ли сюжет именно с ним. Раздел миров представляет альтернативный маршрут: сначала выбирается окружение и цель, затем открывается подходящий персонаж.

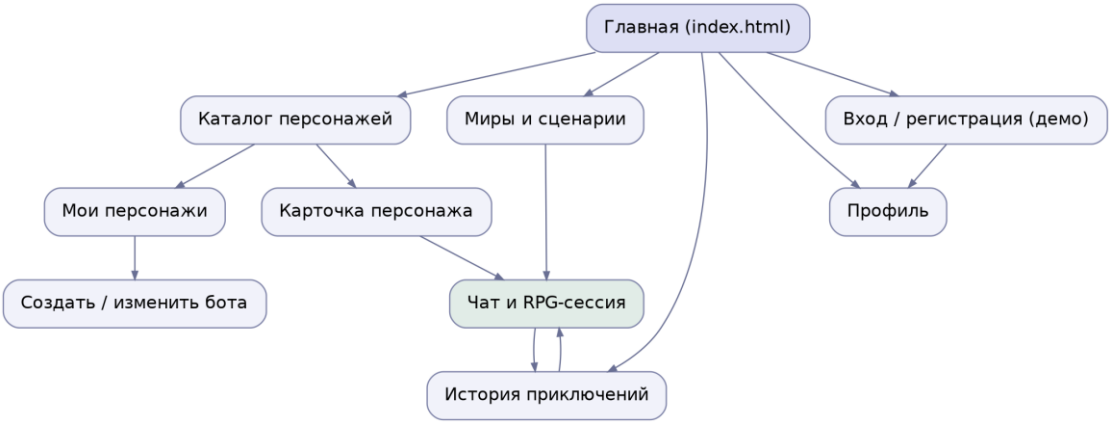


Рисунок 2 – Схема основных переходов между страницами

На схеме показана взаимосвязь десяти страниц приложения. Она не является строгим деревом, поскольку пользователь может открыть чат с нескольких исходных экранов и вернуться в него из истории. Система ссылок должна учитывать идентификаторы объектов: bot.html?id=... предназначен для выбранного персонажа, а chat.html?session=... – для определенного прохождения. Это помогает избежать потери контекста при переходах и повторном открытии адреса.

Карточка бота проектируется как представление одной записи каталога. Помимо имени, изображения и роли на ней выводятся сюжетная завязка, биография, особенности поведения, способности и связь с игровым миром.

Основным действием является переход к переписке. Пользователь не должен искать кнопку запуска среди длинного описания. При неправильном идентификаторе необходимо предусмотреть отдельное информативное состояние и ссылку на возвращение в каталог.

Каталог персонажей строится на повторяемой сетке карточек, которая изменяет число столбцов в зависимости от ширины экрана. В верхней части страницы размещаются поле поиска и тематические фильтры. После ввода запроса карточки перестраиваются без полной перезагрузки. Состояние отсутствия результатов проектируется заранее: вместо пустой области отображается пояснение. Это делает поведение сайта понятным и предотвращает впечатление технической ошибки.

Чат является самым сложным экраном системы, поскольку здесь нужно одновременно отобразить игровой текст, историю предыдущих приключений, характеристики героя, инвентарь, сюжетные действия и поле ввода. На большом экране применяется разделение на три логические области. Центральной остается переписка с ботом, а список сеансов и панель ресурсов занимают вспомогательные боковые части. Роль каждого блока отражена в заголовках и наборе действий.

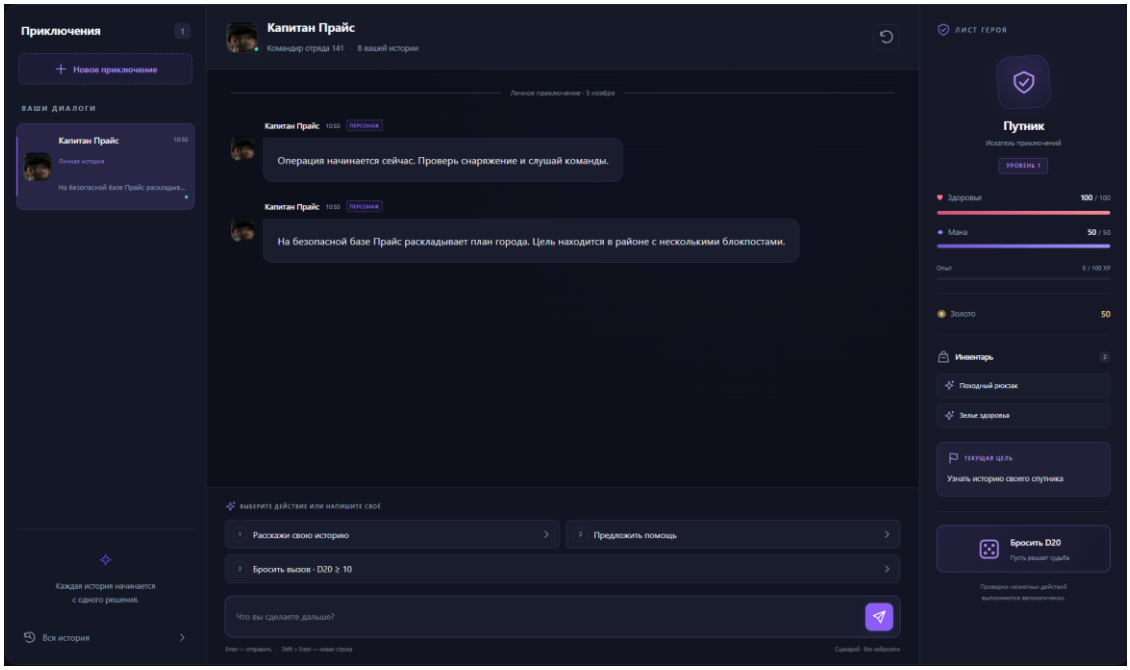


Рисунок 3 – Макет рабочего интерфейса RPG-чата

Для мобильной версии предусмотрен другой принцип организации пространства. Центральная область остается открытой по умолчанию, тогда как список сессий и панель героя доступны через вкладки. Этот подход позволяет не уменьшать размер текста ради одновременного размещения трех колонок. На планшете информационные панели могут располагаться под перепиской. При проектировании важно сохранять доступ к основным действиям даже в случае длинных сообщений и открытой экранной клавиатуры.

В рабочем интерфейсе различаются три вида сообщений: пользовательские, ответы бота и системные. Для каждого вида предлагается собственная форма блока

и визуальная иерархия. Дата и время сообщения занимают второстепенное место, а основной текст должен иметь достаточный интервал между строками. Переходы сюжета и результаты D20 следует показывать отдельно от обычной реплики, чтобы было понятно, каким событием вызвано изменение игровых показателей.

Для конструктора персонажей определена форма с полями биографии, внешности и начальной ситуации, а также область предварительного просмотра. Редактирование собственного бота использует тот же экран, но поля заполняются сохраненными значениями. Личная коллекция разделяет созданных и избранных персонажей. При удалении важно выводить подтверждение, так как эта операция может сделать сохраненные приключения недоступными через обычную навигацию.

История приключений проектируется как набор карточек с именем персонажа, миром, датой обновления и фрагментом переписки. Для каждой сессии предусмотрено возобновление или удаление, для всего списка – очистка с подтверждением. Профиль содержит никнейм, аватар, описание и тему. Экран входа должен обращаться к серверу для регистрации и авторизации, а при ошибках отображать понятное сообщение без сохранения пароля в браузере.

На этапе проектирования учтены состояния нового пользователя, отсутствие избранного и истории, завершенное приключение, недопустимый ввод, истекшая авторизация и временная недоступность сервера. В случае ошибки клиент выводит понятное уведомление и не изменяет состояние героя самостоятельно. При повторном входе пользователь получает прежние сведения из базы данных независимо от того, на каком экране он находится.

При проектировании экранов учитывается объем информации, который появляется по мере развития истории. В начале игры панель характеристик содержит базовые значения, а после нескольких ходов в ней отображаются изменившиеся здоровье, мана, опыт, золото и предметы. Важные изменения не должны теряться среди обычных сообщений. Для этого системные результаты действий визуально отделяются от реплик персонажа, а сведения об инвентаре и состоянии героя остаются доступными независимо от положения переписки.

Макет также определяет состояния интерфейса, которые возникают не только при штатной работе. Для каждой страницы предусматривается отображение загрузки, пустого результата, ошибки запроса и успешного сохранения. Если каталог еще не получен от сервера, пользователь видит нейтральное сообщение о загрузке, а не пустой экран. Если поиск не находит совпадений, интерфейс сообщает об этом и сохраняет введенный запрос, чтобы пользователь мог изменить условия без повторного открытия страницы.

Перед переходом к программной реализации прототип следует проверить на типовых пользовательских маршрутах. Новый посетитель должен найти персонажа, открыть его карточку и понять, как начать диалог. Зарегистрированный игрок должен иметь возможность вернуться к незавершенной сессии через историю. При создании собственного героя пользователь должен видеть обязательные поля и понимать, какие сведения будут показаны в карточке.

3 Разработка веб-приложения

3.1 Создание шаблонов веб-страниц

В качестве основы сохраняются десять HTML-документов: index.html, catalog.html, bot.html, chat.html, create-bot.html, my-bots.html, worlds.html, history.html, profile.html и auth.html. После интеграции прежний демонстрационный экран auth.html должен выполнять настоящую регистрацию и вход через «API». Личная коллекция, профиль, сообщения и история получают данные из базы, а каталог и игровые миры читаются через публичные серверные маршруты.

Страницы используют семантическую разметку HTML5. Внешний вид отделен от разметки посредством CSS, а программное поведение подключено через JavaScript ES-модули. Общие элементы – шапка, подвал, некоторые карточки, диалоги подтверждения и служебные компоненты – описаны в partials/components.html с помощью HTML-элемента template. При открытии страницы JavaScript получает шаблон, клонирует его и подставляет нужные данные.

Использование шаблонов решает проблему дублирования. Если бы одинаковые карточки и формы полностью повторялись в разных файлах, изменение их конструкции потребовало бы исправления нескольких копий. В принятой архитектуре общие элементы поддерживаются в одном месте, а отдельные страницы управляют только своим наполнением. Так, каталог и главная могут использовать общий тип карточки персонажа, хотя набор выводимых объектов различается.

В каждом HTML-документе предусмотрен атрибут data-page, определяющий его программный модуль. Скрипт main.js считывает этот атрибут, загружает шаблоны, формирует общую оболочку и затем импортирует нужную логику. Для главной страницы используется home.js; для остальных имя модуля соответствует функциональному разделу. Такой подход позволяет сохранить единый способ начальной загрузки при множестве независимых маршрутов.

Стили распределены по файлам css/style.css, css/components.css, css/forms.css, css/chat.css и css/responsive.css. Общие настройки включают цветовую палитру и типографику. Компонентные стили определяют поведение карточек, кнопок, меню и уведомлений. Отдельная таблица правил для чата необходима из-за сложного расположения центрального диалога и боковых областей. Адаптивные изменения вынесены в специальный файл для удобства проверки медиаусловий.

Важной частью HTML-архитектуры является отсутствие встроенных сценариев. Обработчики событий определяются в JavaScript, а не в атрибутах onclick. Разметка не включает встроенные style-блоки, и JavaScript не содержит строк с HTML для построения интерфейса. Вместо этого используются cloneNode, replaceChildren, textContent и другие методы DOM API. Разделение делает код проверяемым и снижает риск небезопасной вставки пользовательского текста.

На главном экране отображаются тематический блок, подборки персонажей и игровых миров, а также быстрый запуск приключения. Для зарегистрированного

пользователя дополнительно можно показывать последние сессии, запрашиваемые по «API». При отсутствии токена интерфейс сохраняет доступ к открытым материалам, но защищенные действия переводят посетителя на форму входа.

В каталоге карточки создаются из ответа GET /api/characters с учетом поиска, жанра, сортировки и постраничной навигации. Карточка героя получает сведения через GET /api/characters/:id. Пользовательские персонажи хранятся вместе с официальными, но отличаются признаком владельца и правами изменения. Выбор избранного обращается к защищенным маршрутам, поэтому состояние остается одинаковым после нового входа.

Форма конструктора сохраняет прежние поля и предварительный просмотр, но при отправке должна выполнять POST /api/characters, при редактировании – PUT /api/characters/:id, при удалении – DELETE /api/characters/:id. Сервер проверяет длину полей, ограничение до восьми черт и способностей и принадлежность записи пользователю. Удаление выполняется мягко: персонаж скрывается, а записи старых сессий остаются доступными владельцу.

История загружается запросом GET /api/sessions с поиском и фильтром статуса, а отдельная запись открывается через GET /api/sessions/:id. Профиль изменяется запросом PATCH /api/profile; статистические показатели вычисляет база. Допускается хранение темы оформления как необязательного кэша, но основным источником сведений остается сервер. В интерфейсе необходимо учитывать ситуации отсутствия доступа и устаревшего токена.

В HTML учитываются базовые требования доступности: семантические заголовки, подписи форм, атрибуты alt у значимых изображений, aria-атрибуты управляющих элементов и ссылка для быстрого перехода к основному содержимому. Для навигации активный раздел обозначается aria-current. Эти решения полезны не только пользователям вспомогательных технологий, но и автоматизированной проверке интерфейсов.

В результате структура шаблонов сохраняет единый стиль приложения, но перестает зависеть от локальных пользовательских массивов. Разделение страниц, программных модулей и сетевых методов позволяет одновременно совершенствовать дизайн и безопасность данных. Изменения в существующей верстке ограничиваются обработкой сетевых ответов, ошибками авторизации и состояниями загрузки.

3.2 Реализация динамических элементов на сайте

Динамические элементы остаются реализованными на «JavaScript», однако источником данных становится сервер. Функции из модулей auth.js, catalog.js, bot.js, chat.js, profile.js, history.js и других обращаются к соответствующим маршрутам «/api» через fetch. Файл storage.js больше не предназначен для сохранения персонажей, игровых сессий и профиля. Игровые правила из прежнего rpg-engine.js переносятся в server/services/rpg-engine.js.

Поиск, фильтрация и сортировка персонажей используют параметры запросов GET /api/characters: search, genre, sort, favorites, owner, page и limit. Публичная карточка доступна гостю, а добавление и удаление избранного требует авторизации. После выполнения запроса интерфейс обновляет только нужные компоненты без перезагрузки страницы. Ошибка сети не должна приводить к фиктивному изменению избранного.

Описание шести миров, сорока двух узлов и девяноста сюжетных действий при первоначальном наполнении переносится из файлов «JSON» в таблицы worlds, scenario_nodes и scenario_actions. Публичный запрос возвращает только сведения о мире и главном персонаже. Полное дерево сценария клиенту не передается: он получает текущий узел и доступные варианты после проверки на сервере.

Запуск игры осуществляется запросом POST /api/sessions с идентификаторами персонажа и мира. Сервер проверяет право пользователя, определяет начальную сцену и создает запись сессии вместе с начальными характеристиками и предметами в одной транзакции. Клиенту возвращаются созданный идентификатор, сообщения персонажа, описание текущей цели и параметры героя. Последующие действия ссылаются только на этот идентификатор.

При возвращении в чат выполняется GET /api/sessions/:id. По токену «JWT» сервер определяет владельца и извлекает сообщения, характеристики, инвентарь, статус и доступные варианты действия. Если сессия отсутствует либо принадлежит другому игроку, возвращается код 404 или 403 по принятой политике доступа. В браузере не нужно восстанавливать состояние из «localStorage»; достоверным является ответ сервера.

Игрок может отправить свободный текст по маршруту POST /api/sessions/:id/messages или выбрать действие по POST /api/sessions/:id/actions. Сервер сопоставляет текст с допустимыми ключевыми выражениями или идентификатором действия текущего узла. При неподходящем вводе выдается нейтральная реплика без продвижения сценария. Переход, расход ресурсов и награды вычисляются исключительно на стороне сервера.

Если пользователь пишет фразу, не связанную ни с одним вариантом, приложение выдает одну из резервных реплик и повторяет доступные действия, но не переводит сессию на следующую сцену. Это позволяет свободно вводить сообщения без разрушения логики сценария. Вместе с тем данное поведение ограничивает естественность общения: бот не понимает произвольную речь в смысле нейросетевой модели, а связывает формулировки с заранее заданными действиями.

Случайная проверка D20 выполняется серверным генератором числа от 1 до 20; клиент не может передать готовый результат броска. Для действия с порогом проверяется условие check_threshold, затем применяются effects либо fail_effects и определяется следующий сюжетный узел. Отдельный запрос POST /api/sessions/:id/roll позволяет показать свободный бросок в истории сообщений. Все связанные изменения записываются согласованно.

Для ресурсов действуют отдельные ограничения. Перед выбором действия проверяется достаточно ли маны, золота или другого необходимого ресурса. Если

условие не выполнено, пользователь получает соответствующее пояснение, а текущая сцена не изменяется. При разрешенном действии стоимость списывается и применяются эффекты наград. Значения здоровья и маны ограничиваются установленными верхними и нижними границами.

Опыт, золото и инвентарь составляют прогресс героя. За сюжетные действия могут начисляться очки XP или предметы. При достижении каждого порога в 100 XP увеличивается уровень и частично восстанавливаются HP и MP. В зависимости от сценария игрок может терять здоровье, использовать предметы либо получать новые находки. Если здоровье снижается до нуля, сценарий переводится к неудачному завершению. Поэтому все изменения проходят через общий механизм, а не назначаются каждой кнопкой отдельно.

Специальное действие связано с зельем здоровья. Если в сообщении пользователя обнаружены соответствующие ключевые слова, а в инвентаре есть «Зелье здоровья», приложение восстанавливает до 35 HP и удаляет предмет из коллекции. После использования повторить восстановление тем же предметом невозможно. Это показывает, как команда из естественного текста может менять игровой ресурс при соблюдении условий.

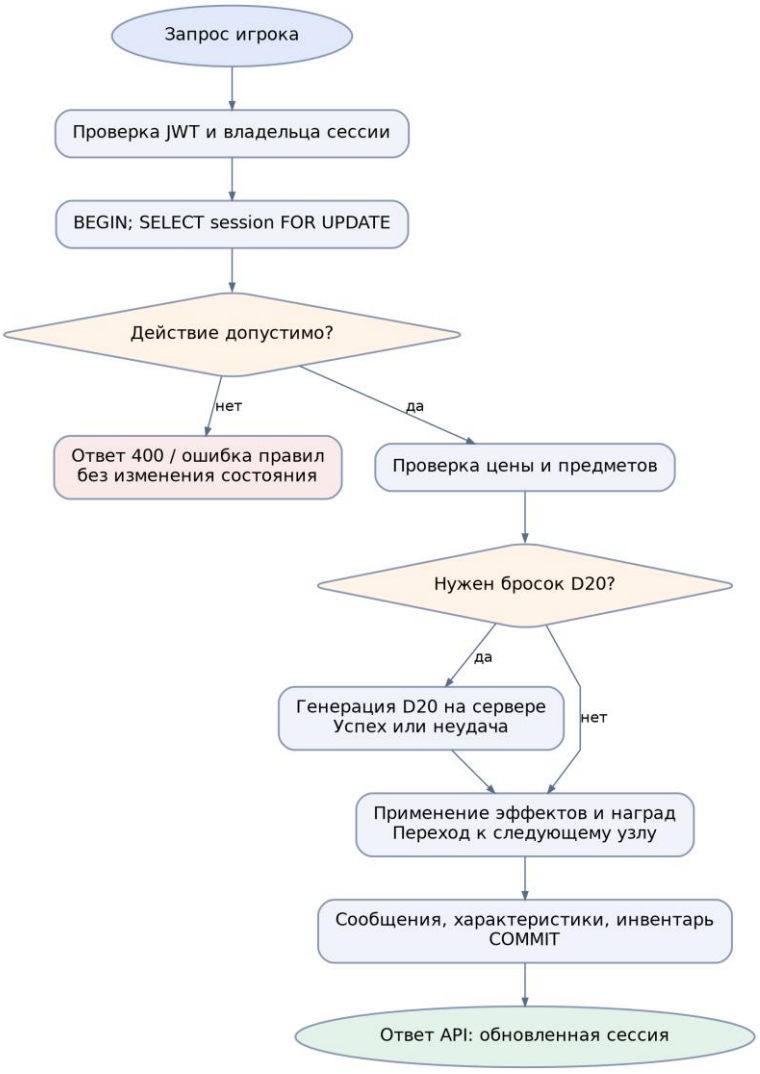


Рисунок 4 – Алгоритм защищенной обработки игрового действия

Обработка одного игрового хода производится в транзакции базы данных. Перед расчетом сервер блокирует запись игровой сессии запросом SELECT ... FOR UPDATE, проверяет принадлежность и допустимость действия, стоимость, необходимые предметы и текущие ресурсы. Затем сохраняет характеристики, предметы, сообщения и новый сюжетный узел. Подтверждение транзакции происходит только после успешного завершения всех операций; при ошибке выполняется откат.

Модульность движка позволяет отдельно проверять определение разрешенного действия, расчет затрат и наград, переходы между сценами, повышение уровня и завершение игры. В новой версии необходимы дополнительные серверные тесты: попытка читать чужую сессию, изменить чужого персонажа, обойти стоимость действия или прислать поддельные характеристики. Результат с сервера является единственным основанием для обновления панели героя.

Уведомления, подтверждения удаления, предварительный просмотр героя, переключение оформления и адаптивные панели сохраняются. Для операций с сервером добавляются состояния ожидания ответа и обработки кодов 400, 401, 403, 404, 409 и 500. При истечении токена защищенные кнопки не отправляют игровые действия повторно без авторизации, а выводят сообщение о необходимости входа.

Для защиты пользовательского ввода применяется textContent, благодаря которому введенные теги отображаются буквально, не превращаясь в исполняемую разметку. Это особенно важно в текстовом чате, где пользователь может вставить произвольные символы. Сценарий не должен зависеть от HTML в сообщении. Подготовленные тестовые файлы включают специальные значения для оценки такого поведения.

Для собственного героя, не связанного с публичным миром, по заданию предусмотрен скрытый мир personal-story. Он содержит универсальную ветвящуюся историю и используется серверным движком с учетом введенного пользователем описания. Данные собственного персонажа хранятся в таблице characters, а его приключения – в game_sessions. Поэтому удаление карточки героя не уничтожает уже сохраненную историю.

3.3 Реализация кода сервера

Техническое задание предусматривает замену сервера статических файлов полноценным серверным приложением на «Node.js 20+» и «Express». Сервер раздает файлы из каталога public и обрабатывает запросы с префиксом «/api». В каталоге server размещаются точка входа index.js, модуль подключения к базе db.js, начальное заполнение seed.js, промежуточные проверки доступа, маршруты и игровой движок. Существующие страницы и оформление сохраняются.

Обращение браузера к серверу выполняется через «fetch API». После получения запроса маршрутизатор проверяет формат данных, наличие токена для

закрытых операций, права пользователя и существование требуемых объектов. Затем вызывается сервис, выполняющий бизнес-правила и запросы в «PostgreSQL». Ответ формируется как «JSON» либо как статический файл. Прямое соединение интерфейса с базой данных не допускается.

Основные модули маршрутов распределены по назначению: auth.js обслуживает регистрацию и вход, profile.js – личные данные, characters.js – каталог и пользовательских героев, worlds.js – игровые миры, favorites.js – избранное, sessions.js – прохождения, admin.js – административные действия. Повторяемая проверка токена и роли вынесена в middleware/auth.js, а вычисления состояния приключения – в services/rpg-engine.js.

Регистрация производится запросом POST /api/auth/register, который принимает никнейм, электронную почту и пароль. Сервер проверяет длину полей и уникальность адреса без учета регистра, формирует хеш пароля функцией «bcrypt» с десятью раундами и создает записи users и profiles одной транзакцией. При входе POST /api/auth/login проверяется пароль, после чего сервер выдает подписанный токен «JWT» и публичные сведения пользователя.

Авторизованный клиент передает токен в заголовке Authorization: Bearer TOKEN. Проверка токена выполняется до доступа к приватным маршрутам. Роль user дает возможность управлять только собственными героями и сессиями; роль admin дополнительно открывает управление пользователями, мирами, узлами и действиями. Серверные проверки выполняются независимо от того, какие кнопки видны в браузере.

Профиль читается через GET /api/profile и редактируется посредством PATCH /api/profile. Каталог героев поддерживает поиск, жанры, сортировку, фильтры принадлежности и избранного, а также пагинацию. Для пользовательских героев применяются запросы POST, PUT и DELETE; последнее действие означает мягкое удаление путем изменения is_active и deleted_at. Официальные карточки может изменять только администратор.

Игровые миры доступны для чтения через GET /api/worlds и GET /api/worlds/:id. Публичный ответ содержит название, жанр, сложность, описание и основного персонажа. Полные массивы сюжетных узлов и эффектов не пересылаются клиенту, чтобы игрок не мог заранее получить все варианты или самостоятельно выбрать следующее состояние.

Список приключений запрашивается через GET /api/sessions с фильтрами и постраничной навигацией. Новая игра создается POST /api/sessions, а текущая сессия читается через GET /api/sessions/:id. Для удаления одной истории служит DELETE /api/sessions/:id, для очистки всей личной истории – DELETE /api/sessions. Маршрут POST /api/sessions/:id/restart создает новое прохождение на основе выбранной истории.

Игровые команды поступают на POST /api/sessions/:id/messages, POST /api/sessions/:id/actions и POST /api/sessions/:id/roll. Серверный движок извлекает текущую сцену из базы, проверяет разрешенные действия, ресурсы и инвентарь, генерирует результат D20 при необходимости и применяет эффекты. Пользователь не может самостоятельно отправить новое значение HP, MP, XP, золота, уровня, инвентаря или статуса.

При выполнении игрового хода открывается транзакция, а запись сессии блокируется оператором SELECT ... FOR UPDATE. Блокировка препятствует одновременному применению двух действий с одной сессией. После проверки условий сервер изменяет характеристики, инвентарь, текущий узел и сообщения. Если операция не может завершиться полностью, транзакция откатывается, что защищает последовательность игрового прогресса.

После успешного игрового действия сервер возвращает публичные сведения персонажа и мира, историю сообщений, характеристики героя, инвентарь, текущую цель, статус, доступные действия и новый узел. Для всех сообщений установлен порядок sequence_number. При достижении финального узла статус меняется на completed либо failed; законченный сценарий не должен продолжаться обычной игровой командой.

Защита сервера включает ограничение длины сообщения до 2000 символов, ограничение размера тела «JSON», параметризованные запросы, фильтрацию ошибок и исключение password_hash из ответов. Все ошибки «API» имеют единый объект вида {"error":"Описание ошибки"}. Код 401 применяется для отсутствующей авторизации, 403 – для недостаточных прав, 404 – для отсутствующего объекта, 409 – для конфликта, 400 – для некорректного ввода.

Режим администратора основан на отдельной роли. После проверки middleware requireAdmin администратор может просматривать и блокировать учетные записи, управлять официальными героями и игровыми мирами, редактировать узлы и варианты действий, получать сводную статистику. Добавление специальной страницы admin.html допускается, но сервер обязан запрещать вызовы административных маршрутов обычному пользователю.

Конфигурационные параметры, включая DATABASE_URL, JWT_SECRET и пароль к базе, размещаются в переменных окружения. Контейнер приложения публикует порт 3000, а сервер базы допускает подключение с локального компьютера только через 127.0.0.1. В «Docker Compose» сервис app должен запускаться после успешного healthcheck службы database.

Предусмотрены два способа запуска: docker compose up --build для полной контейнерной среды и локальный запуск npm install, npm run seed, npm start с отдельно запущенным контейнером базы. Файл start.bat выполняет запуск и открывает http://localhost:3000. В документации требуется описать параметры окружения, первоначальное заполнение, тестовые учетные записи, команды проверки и устранение типичных ошибок подключения.

Файл server/seed.js должен создавать начальные таблицы после миграции либо использовать готовую схему, импортировать 12 героев, 6 публичных миров, 42 сюжетных узла, 90 действий и скрытый мир personal-story. Дополнительно предусматриваются тестовые пользователи, избранные персонажи и не менее двух сессий. Повторный запуск начального заполнения не должен создавать дубликаты, а тестовые пароли должны храниться только в виде хешей.

Раздел описывает целевую серверную архитектуру в точном соответствии с предоставленным техническим заданием. Исходный ZIP-файл содержит прежний локальный сервер serve.mjs; обновленные серверные исходники в переданных материалах не представлены. Поэтому описание маршрутов и транзакций является

проектной документацией, а не утверждением об уже пройденном испытании готовой серверной сборки.

Перенос игрового движка на сервер обеспечивает проверку неизменности игровых правил и защищает от подмены состояния на стороне браузера. В результате «RPG Nexus» должен стать системой, где интерфейс отвечает за отображение и ввод, сервер – за аутентификацию и игровую логику, а «PostgreSQL» – за целостное хранение истории.

3.4 Реализация базы данных

Для новой версии «RPG Nexus» выбрана реляционная система управления базами данных «PostgreSQL 16». База rpg_nexus должна хранить пользователей, профили, игровых персонажей, миры, сюжетные узлы и действия, избранное, сессии, сообщения и инвентарь. Такая модель заменяет локальное хранение изменяемых пользовательских объектов и позволяет организовать связи через внешние ключи, ограничения и транзакции.

Всего предусмотрено десять связанных таблиц. users хранит адрес электронной почты, хеш пароля, роль, активность и даты учетной записи. profiles содержит никнейм, аватар, биографию и тему; ее первичный ключ одновременно является внешним ключом на users, что создает отношение один к одному. Адрес электронной почты должен быть уникальным без учета регистра благодаря индексу на lower(email).

Таблица worlds описывает игровые миры, включая название, жанр, сложность, текстовые поля, изображение и признаки публичности. Таблица characters хранит официальных и пользовательских героев, их владельца и связанный мир, внешность, описание, биографию, приветствие, черты, способности и активность. Для пользовательского персонажа owner_id обязателен; официальный герой принадлежит системе. Для удаляемых героев сохраняется запись и история с помощью мягкого удаления.

Разделение сведений по отдельным таблицам позволяет избежать многократного повторения одних и тех же данных. Учетная запись хранится независимо от профиля, а описание персонажа отделено от конкретной игровой сессии. Благодаря этому один герой может использоваться в нескольких приключениях, а изменение его текущего описания не должно переписывать уже сохраненную историю. Внешние ключи обеспечивают связь объектов, а ограничения и транзакции защищают данные от частичного обновления.

Дополнительно модель должна обеспечивать однозначное восстановление порядка событий. Сообщения получают последовательные номера внутри сессии, а время создания сохраняется отдельно. При чтении истории сервер сортирует записи по номеру, что позволяет корректно восстановить диалог даже при большом количестве сообщений.

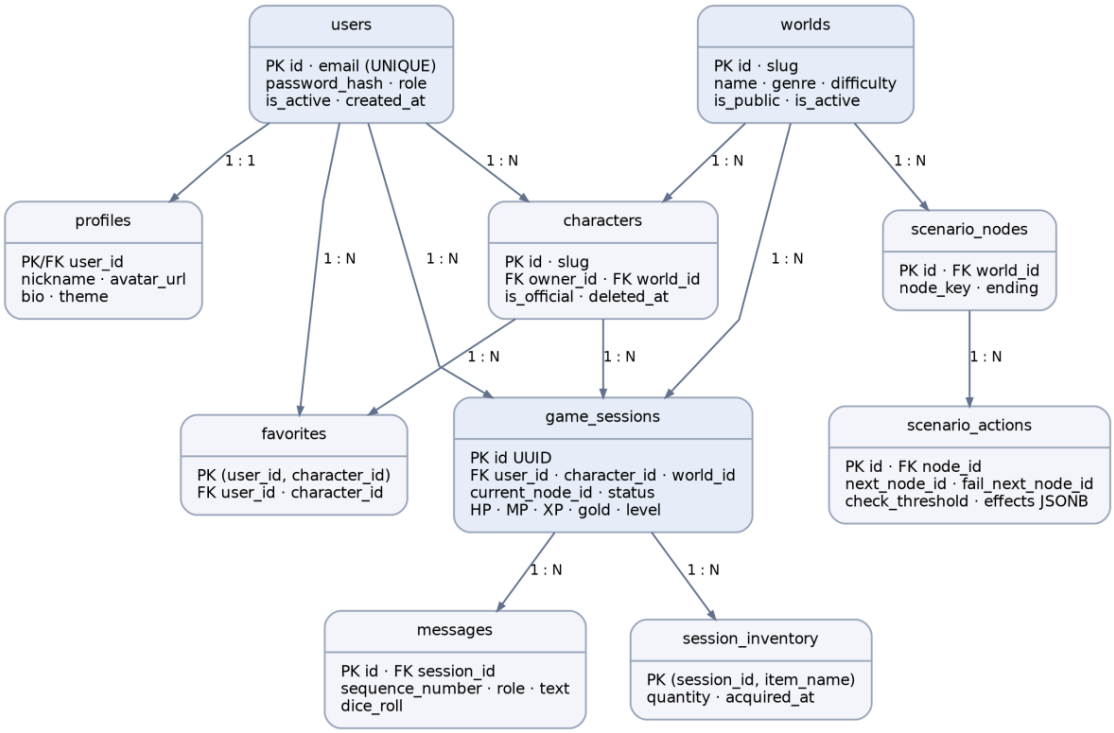


Рисунок 5 – Логическая схема базы данных «RPG Nexus» (10 таблиц)

На рисунке 5 показана схема десяти таблиц «PostgreSQL» с первичными и внешними ключами. Пользователь связан с одним профилем, множеством созданных персонажей и сессий. Избранное реализует связь многие ко многим между users и characters через составной ключ. Мир связан с героями и сценарными узлами, каждый узел – с действиями, а сессия – с сообщениями и предметами инвентаря.

Таблица favorites содержит составной ключ (user_id, character_id), поэтому повторная запись одного героя в избранное одного игрока невозможна. scenario_nodes хранит ключ, текст, цель и признак финала, причем пара (world_id, node_key) уникальна. Таблица scenario_actions включает ссылки на обычный и неуспешный переход, порог D20, обязательный предмет, ключевые слова и эффекты в типе «JSONB».

game_sessions использует первичный ключ типа «UUID» и содержит владельца, персонажа, мир, активную сцену, статус, уровень, здоровье, ману, опыт, золото, цель и временные метки. Дополнительные снимки имени героя, персонажа, аватара и мира позволяют показывать сохраненную историю после изменения пользовательской карточки. Статусы ограничиваются значениями active, completed и failed; числовые характеристики не должны быть отрицательными.

Сообщения сессий записываются в таблицу messages с внешним ключом session_id, порядковым номером, ролью, текстом и необязательным значением броска D20. Пара (session_id, sequence_number) уникальна, что сохраняет порядок переписки. Таблица session_inventory содержит составной ключ (session_id, item_name), количество предметов больше нуля и дату получения; при удалении сессии ее сообщения и предметы удаляются каскадно.

При проектировании предусмотрены ограничения ссылочной целостности: ON DELETE CASCADE для зависимых профилей, избранного, сообщений и инвентаря; ON DELETE RESTRICT там, где необходимо сохранить историю сессии; ON DELETE SET NULL для допустимого отключения связи персонажа с миром. Удаление обычного персонажа реализуется не физическим удалением строки, а изменением is_active и deleted_at. Это сохраняет корректность исторических отчетов.

Для ускорения запросов создаются индексы characters.owner_id, characters.world_id, characters.genre и characters.popularity, favorites.user_id, scenario_nodes.world_id, scenario_actions.node_id, game_sessions по user_id с updated_at и по status, messages по session_id с sequence_number. Помимо этого используется уникальный индекс по нижнему регистру электронной почты. Индексы помогают фильтровать каталог, историю и сообщения без избыточного перебора записей.

Начальное заполнение выполняет server/seed.js. В базу должны быть импортированы 12 официальных персонажей, 6 публичных миров, 42 сценарных узла и 90 действий, а также скрытый мир personal-story. Процедура начальной загрузки выполняется транзакционно, должна быть повторяемой и создавать тестовые данные без дубликатов. Основной код приложения обращается к таблицам параметризованными запросами через «pg», а не хранит рабочие сведения в «localStorage».

Ограничения значений определяются на уровне приложения и схемы: роли user/admin, темы dark/light, сложности «Начальный», «Средний», «Сложный», статус сессии active/completed/failed и роль сообщения user/bot/system. Для текстовых полей применяются ограничения длины. Числовые показатели уровня и ресурсов проверяются перед сохранением, а количество предметов всегда должно быть положительным.

Для корректной игровой транзакции приложение получает запись game_sessions с блокировкой FOR UPDATE, извлекает сценарное действие и ресурсные требования, затем изменяет записи game_sessions, messages и session_inventory как единую операцию. При ошибке все запросы откатываются. Это предотвращает ситуацию, когда золото списано, а переход в следующий сюжетный узел не выполнен.

В проекте предусмотрено вычисление статистики профиля SQL-запросами, а не хранение произвольного счетчика. Количество созданных героев определяется из characters, избранных – из favorites, завершенных приключений – из game_sessions со статусом completed, общее число игр – из всех сессий пользователя. Такой способ исключает рассогласование счетчиков с фактическими данными.

Архитектура развертывания предусматривает постоянный том базы данных «Docker», отдельного пользователя базы и передачу пароля через переменные окружения. Инициализация демонстрационных записей не должна выполняться автоматически при каждом старте рабочей системы, чтобы не сбрасывать пользовательский прогресс. Команда npm run seed рассматривается как отдельное действие для учебного наполнения.

Таким образом, выбранная структура поддерживает учетные записи, совместное использование официального каталога и независимые личные истории. Ее описание составлено по техническому заданию на серверную доработку; фактическое создание таблиц и проверка запросов должны быть подтверждены после получения новой серверной версии проекта.

Для повышения целостности данных в таблицах применяются ограничения, соответствующие смыслу хранимых значений. Адрес электронной почты пользователя является уникальным без учета регистра, количество здоровья, маны, опыта и золота не может быть отрицательным, а статус сессии ограничивается предусмотренными состояниями. Внешние ключи связывают сообщения с игровыми сессиями, а сессии – с пользователями, персонажами и мирами. При удалении учетной записи зависимые личные данные обрабатываются согласно выбранной политике удаления.

Для ускорения часто используемых запросов предусматриваются индексы по идентификатору владельца, времени создания и ключам связей. История приключений выбирается по пользователю и сортируется по дате последнего изменения, поэтому соответствующий индекс уменьшает объем просматриваемых записей. Сообщения одной сессии выбираются по идентификатору сессии и порядковому номеру. При этом индексы не заменяют проверку прав доступа: сервер включает идентификатор текущего пользователя в запрос к личным объектам.

Начальное заполнение базы должно выполняться отдельно от обычного запуска приложения. Справочные записи официальных персонажей, миров и сценарных действий добавляются при подготовке новой базы или специальной командой заполнения. Повторное выполнение такой команды не должно создавать дубликаты и не должно удалять личные записи пользователей. Разделение демонстрационных данных и пользовательского прогресса особенно важно при обновлении проекта, поскольку новые версии каталога не должны сбрасывать уже начатые приключения.

При изменении структуры базы следует применять последовательные миграции, позволяющие обновлять схему без случайного удаления пользовательских данных. Перед крупным обновлением создается резервная копия, после чего проверяются ограничения, связи и основные запросы. Такой порядок особенно важен для проекта с длительными игровыми сессиями: сохраненная история, инвентарь и характеристики должны оставаться согласованными даже после изменения состава таблиц.

После обновления схемы необходимо проверить не только успешность миграции, но и чтение ранее созданных записей. Контрольные запросы должны подтвердить, что профиль связан с нужной учетной записью, сообщения сохраняют исходный порядок, а состав инвентаря соответствует последнему состоянию приключения.

4 Тестирование веб-приложения

Цель тестирования расширенной версии – подтвердить корректность интерфейса, сервера и базы данных. Проверка должна охватывать прежние игровые сценарии, регистрацию, разграничение доступа, сохранение прогресса в «PostgreSQL», согласованность транзакций и защиту от подмены игровых характеристик. Для точности в настоящем отчете результаты тестов исходного клиента отделены от обязательных проверок нового серверного компонента.

Исходный архив включает автоматические проверки игровой логики и разделения HTML, CSS и JavaScript; их результаты относятся к прежней клиентской версии. Новое техническое задание предусматривает отдельные тесты «REST API» с подготовленной базой и тестовыми учетными записями. Сценарии серверных проверок должны запускаться повторяемо, с контролем ожидаемых HTTP-кодов и реального состояния таблиц после операции.

Для предоставленного ZIP-архива тесты были фактически запущены в Node.js версии 22.16.0. Результат выполнения: 11 успешных тестов игрового движка и 4 успешных теста разделения кода, всего 15. Ошибок или пропущенных тестов в этих двух наборах не обнаружено. Данный итог относится только к покрытым алгоритмам и структуре и не означает автоматического подтверждения визуальной корректности всех страниц на любых устройствах.

Для серверной части требуется проверить регистрацию нового игрока, ошибку повторного адреса, вход с правильным и неправильным паролем, получение текущего пользователя, изменение профиля и запрет доступа без токена. Проверки должны подтвердить отсутствие исходного пароля в базе, уникальность адреса без учета регистра и защищенность маршрутов по роли пользователя.

Для каталога и личных данных проверяются получение миров, создание пользовательского героя, редактирование только собственной карточки, мягкое удаление и сохранение старой истории, добавление и исключение избранного. Отдельно проверяются ограничения длины формы, не более восьми характеристик и способностей, фильтры каталога и корректность прав администратора.

Игровые тесты должны пройти создание сессии и начального инвентаря, отправку текста, выбор действия, генерацию D20, начисление опыта и золота, изменение HP и MP, использование зелья, повышение уровня и получение обоих вариантов финала. Для каждого хода необходимо убедиться, что сообщения, инвентарь и характеристики записаны одной транзакцией, а неподходящее действие не меняет состояние.

Обязательны негативные сценарии: запрет чтения чужой сессии и редактирования чужого героя, отказ принять поддельный результат D20 или значения ресурсов, проверка недостатка золота и маны, повторной записи избранного и неподходящего статуса. Дополнительная нагрузочная проверка должна показать, что параллельные команды одной сессии не обходят блокировку SELECT FOR UPDATE.

Интерфейсные тесты охватывают все десять сохраненных страниц на ширинах 320, 768 и 1440 пикселей. Проверяются переходы, загрузка изображений

и API-данных, отсутствие горизонтальной прокрутки, формы авторизации, запуск приключения, история и профиль. После обновления страницы должны восстанавливаться серверные данные. Тестирование осуществляется при доступном работающем сервере и базе.

Браузерный сценарий прежнего архива ранее столкнулся с сетевым ограничением Chromium в тестовой среде. Поэтому успешную проверку новых серверных маршрутов и базы считать подтвержденной нельзя: в предоставленных материалах имеются техническое задание и исходная клиентская версия, но отсутствует результат запуска доработанного backend. Для приемки требуется повторить испытания на обновленной сборке и зафиксировать протокол.

План приемки включает запуск контейнеров app и database, проверку состояния healthcheck, регистрацию тестового игрока, создание приключения, изменение состояния и чтение его после нового входа. Следует проверить наличие десяти таблиц, импорт начальных данных и сохранность записей после перезапуска контейнеров. Для документации нужны фактические скриншоты выполнения, вывод тестов и подтверждение корректных связей в базе.

Отдельно проверяются серверная валидация и обработка конфликтов: пустые поля, недопустимая длина текста, повторный email, неправильный пароль, доступ к чужим данным, неподдерживаемые действия и недоступная база. Сервер должен выдавать стандартизованную ошибку без раскрытия служебной информации. На клиенте сообщение об ошибке должно быть понятным и не приводить к ложной смене игрового статуса.

Нефункциональная проверка включает отсутствие секретов и паролей в коде браузера, использование параметризованных запросов, ограничение тела «JSON», сохранение данных в постоянном томе, производительность чтения длинной истории и восстановление после перезапуска. Также необходимо убедиться, что демонстрационное наполнение не удаляет пользовательские данные при обычном запуске приложения.

Итог: ранее подтверждено прохождение 15 автоматических тестов исходного клиентского архива. Полный набор проверок новой серверной версии, включая регистрацию, «PostgreSQL», защищенные маршруты и транзакции, описан в техническом задании и подлежит выполнению после внедрения новых файлов. Такое разделение не позволяет ошибочно считать проектирование базы доказательством ее фактической работоспособности.

Особое внимание уделяется повторному выполнению операций и восстановлению после сбоев. Повторная отправка одного действия не должна дважды списывать ресурсы или начислять награду. Если соединение с базой прерывается во время обработки хода, транзакция должна быть отменена, а состояние сессии оставаться согласованным. После перезапуска приложения пользователь должен получить прежнюю историю и характеристики героя. Результаты ручных и автоматических проверок фиксируются в протоколе с указанием условий выполнения.

Заключение

В ходе практической работы рассмотрено расширение веб-приложения «RPG Nexus», предназначенного для текстовых ролевых приключений и общения с виртуальными персонажами. Исходный проект содержит многостраничный игровой интерфейс и систему сценарных действий. На основании нового технического задания подготовлено описание серверной архитектуры и базы данных, необходимой для централизованного сохранения профилей, персонажей и игровой истории.

При модернизации сохраняются десять HTML-страниц, двенадцать официальных героев, шесть публичных миров, сорок два сюжетных узла и девяносто действий. Функциональные возможности включают поиск, личных персонажей, избранное, чат, историю, броски D20, инвентарь и характеристики игрока. Для перехода от локальной версии к многопользовательской предусмотрены настоящая регистрация, разграничение доступа и хранение личных данных на сервере.

Ключевое архитектурное решение – перенос игровых вычислений из браузера в «Node.js» и «Express». Сервер должен проверять принадлежность сессии, разрешенность действия, ресурсы, случайный результат D20, начисление наград и завершение сценария. Все изменения одного хода выполняются транзакционно в «PostgreSQL» с блокировкой активной сессии, что защищает игровые результаты от случайных конфликтов и подмены со стороны клиента.

Разработана информационная модель из десяти таблиц: users, profiles, worlds, characters, favorites, scenario_nodes, scenario_actions, game_sessions, messages и session_inventory. На диаграмме отражены отношения один к одному, один ко многим и многие ко многим, составные и внешние ключи. Модель учитывает мягкое удаление персонажей, сохранение исторических названий, повторяемое заполнение справочников и связь сообщений с конкретным приключением.

В новой версии «localStorage» не должен выступать основным хранилищем пользовательского прогресса. Содержимое игровых миров и исходного каталога используется для начального заполнения базы, а профили, собственные персонажи, избранное, сообщения и игровые сессии обслуживаются защищенным «API». Благодаря этому данные пользователя можно восстановить после нового входа независимо от устройства при доступном сервере.

При работе с исходным архивом ранее успешно завершились пятнадцать автоматических проверок игрового движка и архитектуры клиентской части. Новое техническое задание дополнительно определяет тесты авторизации, прав доступа, игровых транзакций, запрета изменения чужих объектов и сохранности данных. Эти проверки нельзя считать пройденными без фактически собранной серверной версии, поэтому они включены как обязательный этап приемки.

Важное ограничение состоит в том, что в переданных файлах представлен текст требований к модернизации, а не уже развернутая база «PostgreSQL» и серверные модули. Поэтому подготовленные схемы, маршруты и структура таблиц описывают проектную реализацию. Для окончательного вывода о

работоспособности необходимо получить обновленный код, выполнить миграции и тесты и проверить систему через браузер.

Дальнейшие действия определяются техническим заданием: создать серверные маршруты, хеширование паролей и «JWT»-проверки, перенести игровые правила, разработать таблицы и индексы, заполнить справочники, обеспечить контейнерный запуск и провести автоматические проверки. После выполнения этих операций можно документально подтвердить переход от локального прототипа к полноценной клиент-серверной системе.

Подготовленные в отчете материалы систематизируют требования к интерфейсу, алгоритмам, хранению и тестированию. Диаграмма прецедентов выделяет гостя, игрока и администратора; функциональная модель показывает прием команды, проверку и сохранение действия; информационная модель связывает десять таблиц. Все схемы относятся к тематике «RPG Nexus» и согласованы с предоставленным техническим заданием.

Таким образом, выполнено проектирование серверной модернизации веб-приложения «RPG Nexus» и актуализированы соответствующие разделы документации. Исходные игровые возможности сохранены в техническом описании, а требования к «Express», «PostgreSQL», авторизации, безопасности и транзакциям отражены как состав дальнейшей разработки. Факт завершения реализации и тестирования серверной части должен подтверждаться ее исходными файлами и протоколом проверки.

Важным результатом проектирования является разделение ответственности между интерфейсом, серверной логикой и хранилищем. Клиентская часть отвечает за удобное представление информации и передачу команд, сервер проверяет права и правила игры, а реляционная база обеспечивает целостность и долговременное хранение данных. Такое распределение снижает риск несогласованного состояния и позволяет развивать отдельные компоненты без полной переработки приложения.

Подготовленная документация может использоваться как основа для дальнейшей реализации и проверки проекта. По ней можно последовательно создать структуру базы, настроить маршруты, перенести игровые вычисления на сервер, подключить формы и проверить сохранение данных. Для завершения разработки необходимо сопоставить каждый пункт технического задания с реализованной функцией и приложить подтверждение прохождения соответствующего сценария. Это позволит отделить уже описанные проектные решения от тех возможностей, которые будут подтверждены только после запуска обновленной версии.

Таким образом, результатом работы является целостное описание целевой версии «RPG Nexus»: определены основные роли пользователей, состав страниц, правила взаимодействия, структура хранения данных и подход к проверке. Сформированные материалы позволяют продолжить разработку последовательно, начиная с базы и серверных маршрутов и заканчивая приемочными испытаниями пользовательских сценариев. При реализации важно сохранить согласованность между техническим заданием, программным кодом и приложенными схемами.

Список используемых источников

1 Сайт о программировании Metanit.com [Электронный ресурс]. – Режим доступа: https://metanit.com/web (дата обращения: 19.10.2026).

2 Современный учебник JavaScript [Электронный ресурс]. – Режим доступа: https://learn.javascript.ru (дата обращения: 20.10.2026).

3 Интерактивный тур по использованию системы контроля версий Git How To [Электронный ресурс]. – Режим доступа: https://githowto.com/ru (дата обращения: 21.10.2026).

4 Инструментарий для интерфейса Bootstrap [Электронный ресурс]. – Режим доступа: https://getbootstrap.com/docs/5.3/getting-started/introduction (дата обращения: 22.10.2026).

5 Yandex Practicum – Адаптация сайта для мобильных устройств [Электронный ресурс]. – Режим доступа: https://practicum.yandex.ru/blog/kak-adaptirovat-sayt-pod-mobilnye-ustroystva (дата обращения: 23.10.2026).

6 JSON Web Tokens – документация [Электронный ресурс]. – Режим доступа: https://jwt.io/introduction (дата обращения: 24.10.2026).

7 Лаборатория Касперского. Что такое SSL-сертификат [Электронный ресурс]. – Режим доступа: https://www.kaspersky.ru/resource-center/definitions/what-is-a-ssl-certificate (дата обращения: 25.10.2026).

8 Стратегия Lazy loading [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/Performance/Guides/Lazy_loading (дата обращения: 27.10.2026).

9 Что такое SEO и из чего оно состоит [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/900968/ (дата обращения: 28.10.2026).

10 GitHub – веб-сервис для хостинга IT-проектов [Электронный ресурс]. – Режим доступа: https://github.com (дата обращения: 29.10.2026).

11 GitKraken – визуализация сложных наборов данных [Электронный ресурс]. – Режим доступа: https://www.gitkraken.com/ (дата обращения: 30.10.2026).

12 WebReference – справочник HTML и CSS [Электронный ресурс]. – Режим доступа: https://webref.ru/ (дата обращения: 01.11.2026).

13 Pinterest – социальный интернет-сервис и фотохостинг [Электронный ресурс]. – Режим доступа: https://ru.pinterest.com (дата обращения: 02.11.2026).

14 Консорциум «Кодекс» – фонд нормативно-технических документов [Электронный ресурс]. – Режим доступа: https://docs.cntd.ru/ (дата обращения: 04.11.2026).

15 Руководство по jQuery [Электронный ресурс]. – Режим доступа: https://jquery.com (дата обращения: 05.11.2026).

Приложение А
(обязательное)

Входные документы

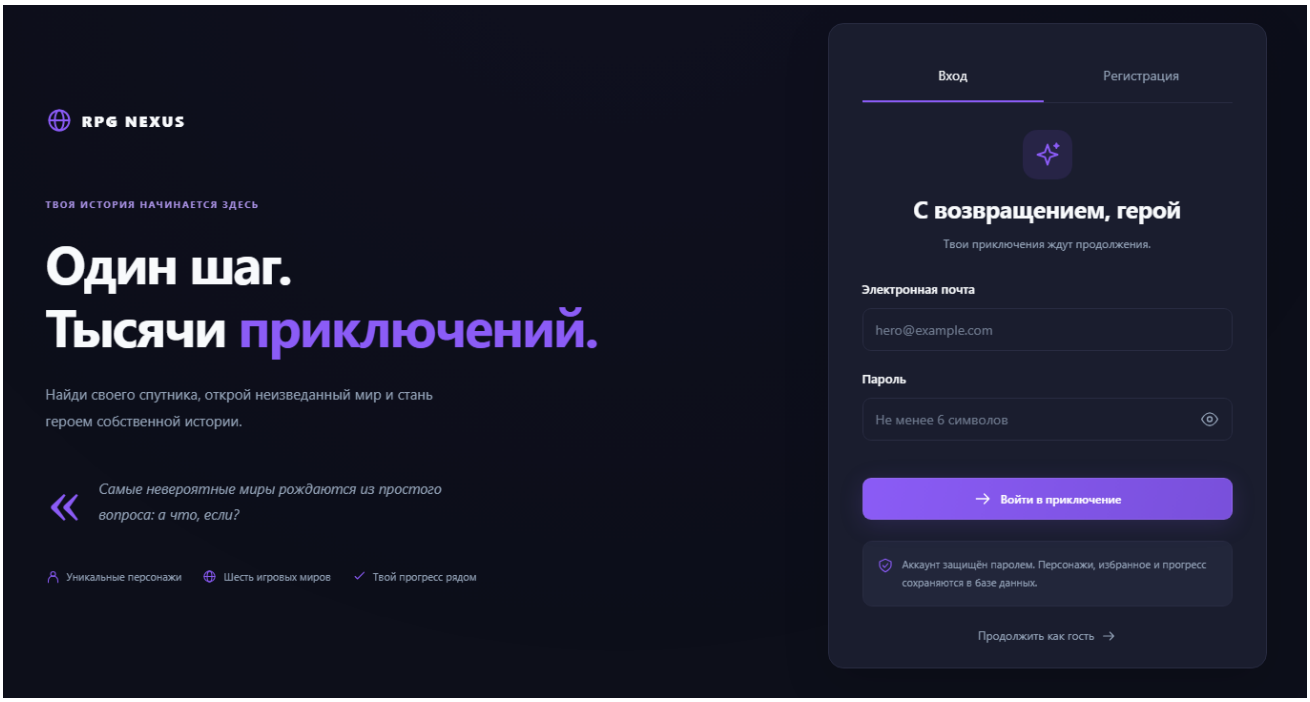


Рисунок А.1 – Входные документы сайта

Приложение Б
(обязательное)

Выходные документы

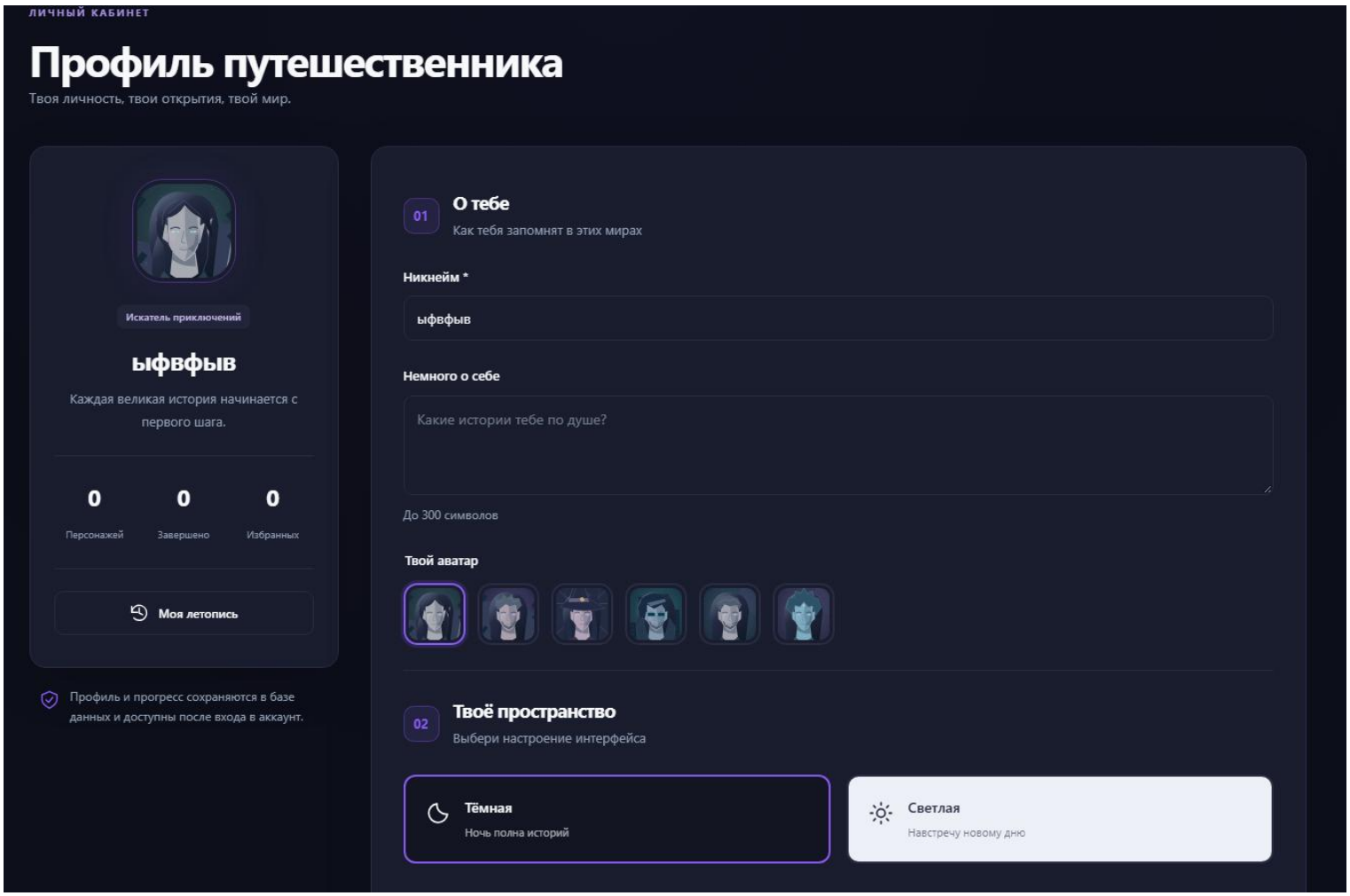


Рисунок Б.2 – Выходные документы сайта

Приложение В
(обязательное)

Информационная модель

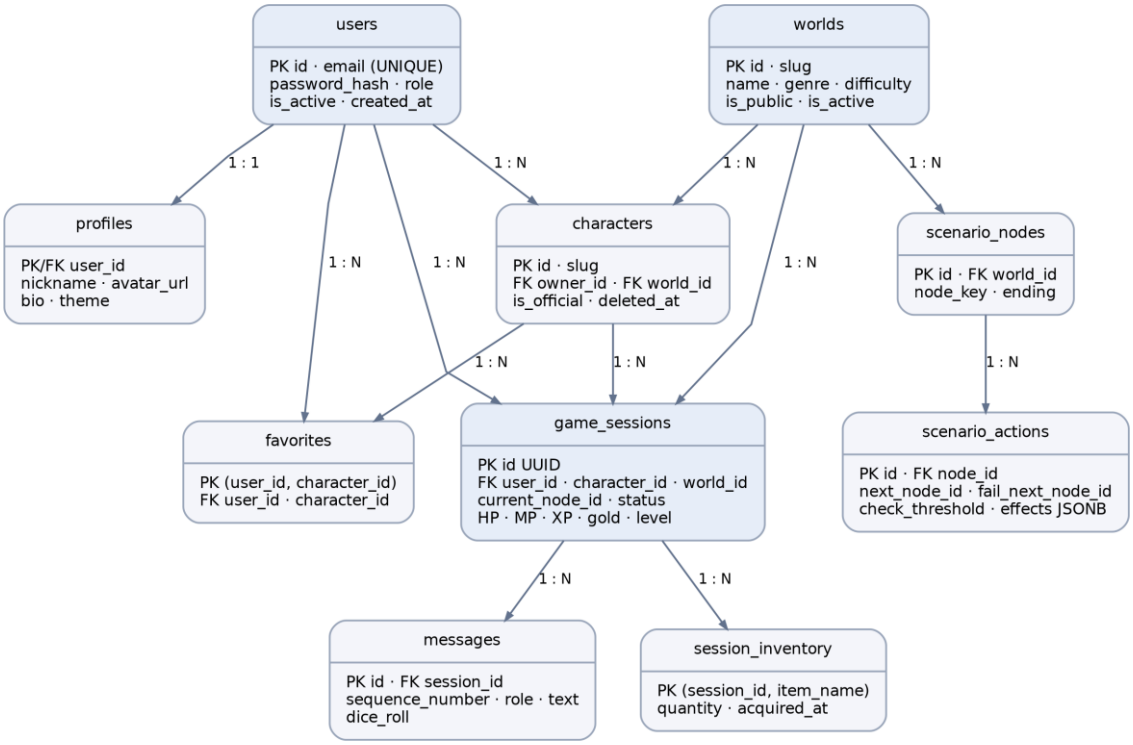


Рисунок В.1 – Информационная модель базы «PostgreSQL»

Приложение Г
(обязательное)

Функциональная модель



Рисунок Г.1 – Функциональная модель серверной обработки

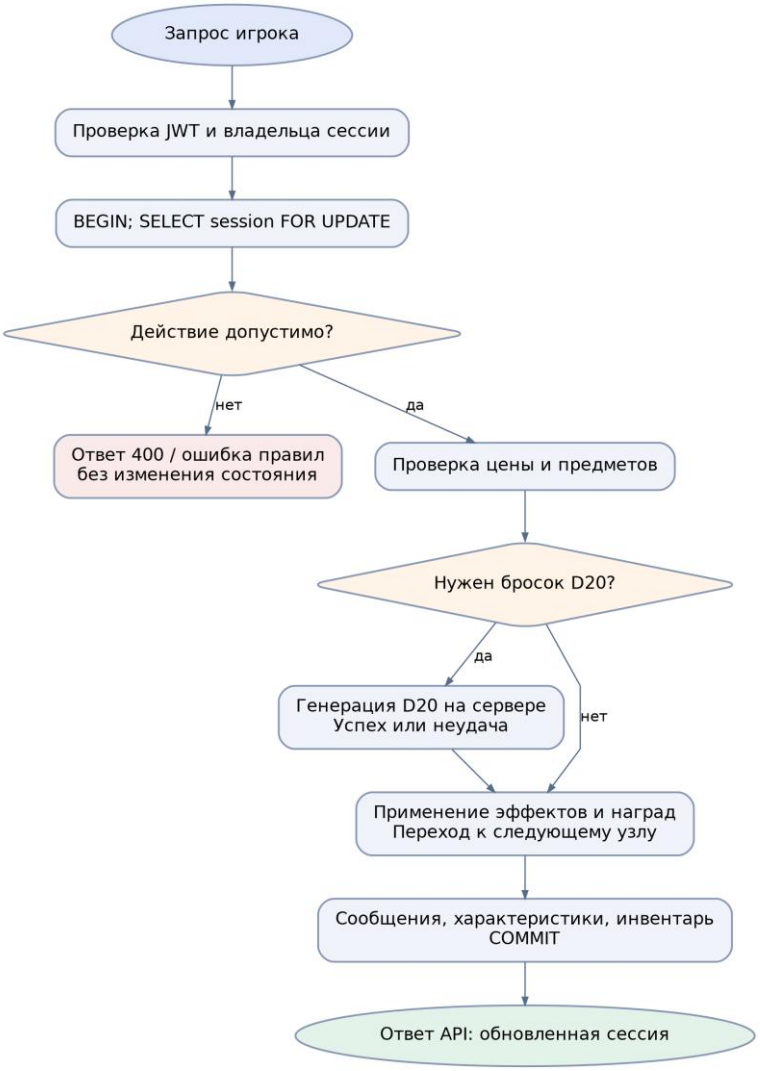


Рисунок Г.2 – Алгоритм сохранения игрового хода

Приложение Д
(обязательное)

Диаграмма прецедентов

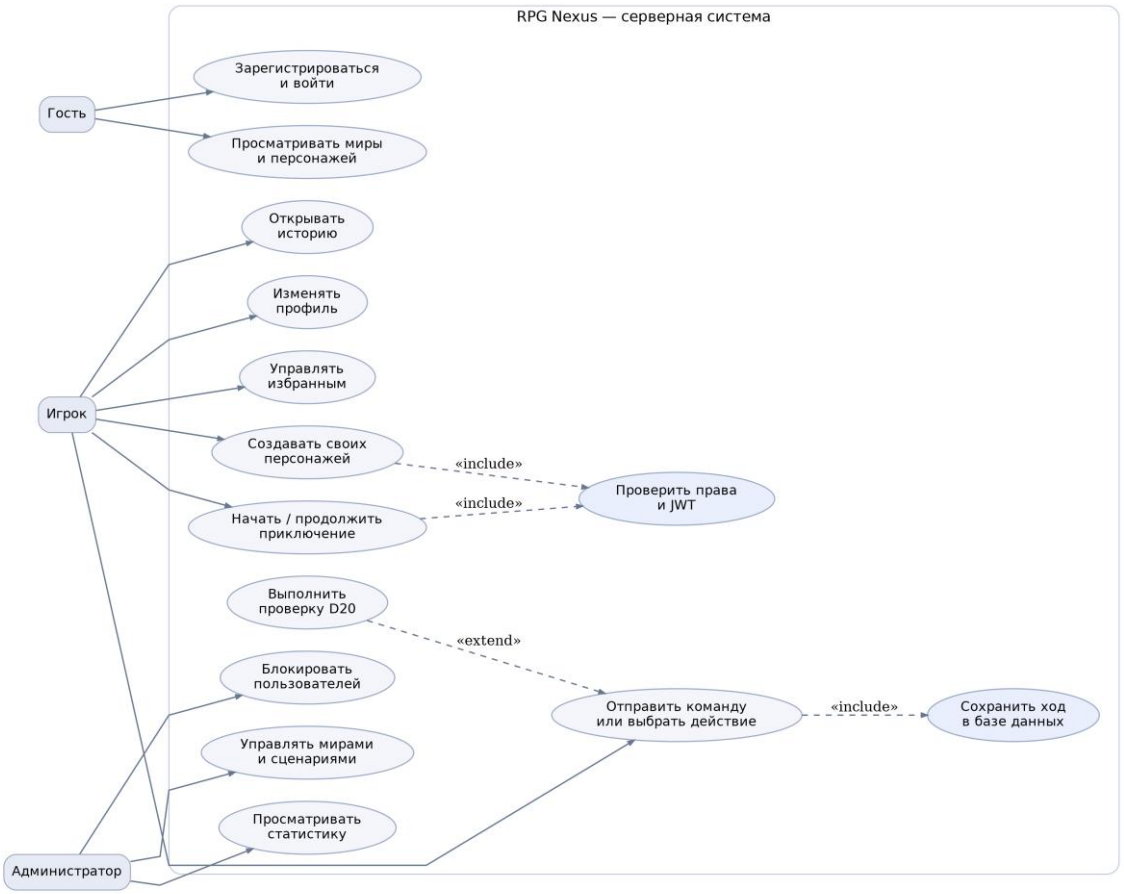


Рисунок Д.1 – Диаграмма прецедентов веб-приложения «RPG Nexus»

Приложение Е
(обязательное)

Дизайн сайта

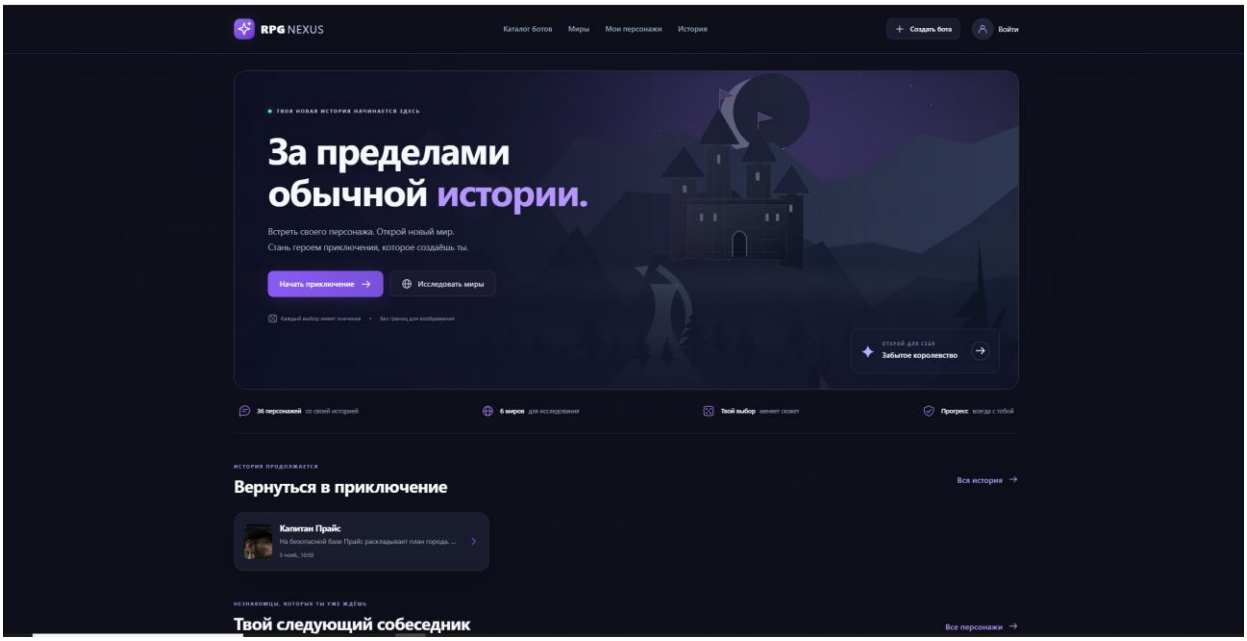


Рисунок Е.1 – Главная страница сайта

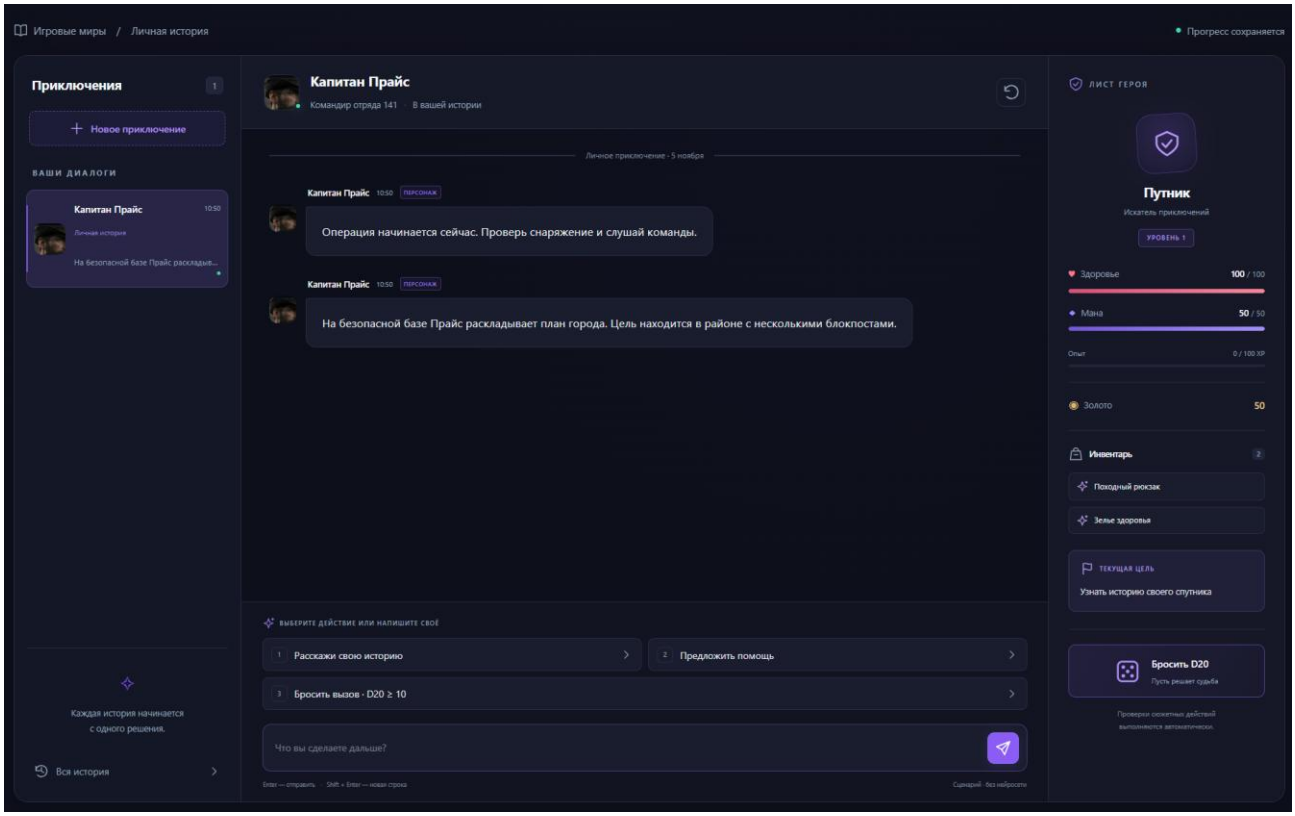


Рисунок Е.2 – Чат – интерфейс сайта

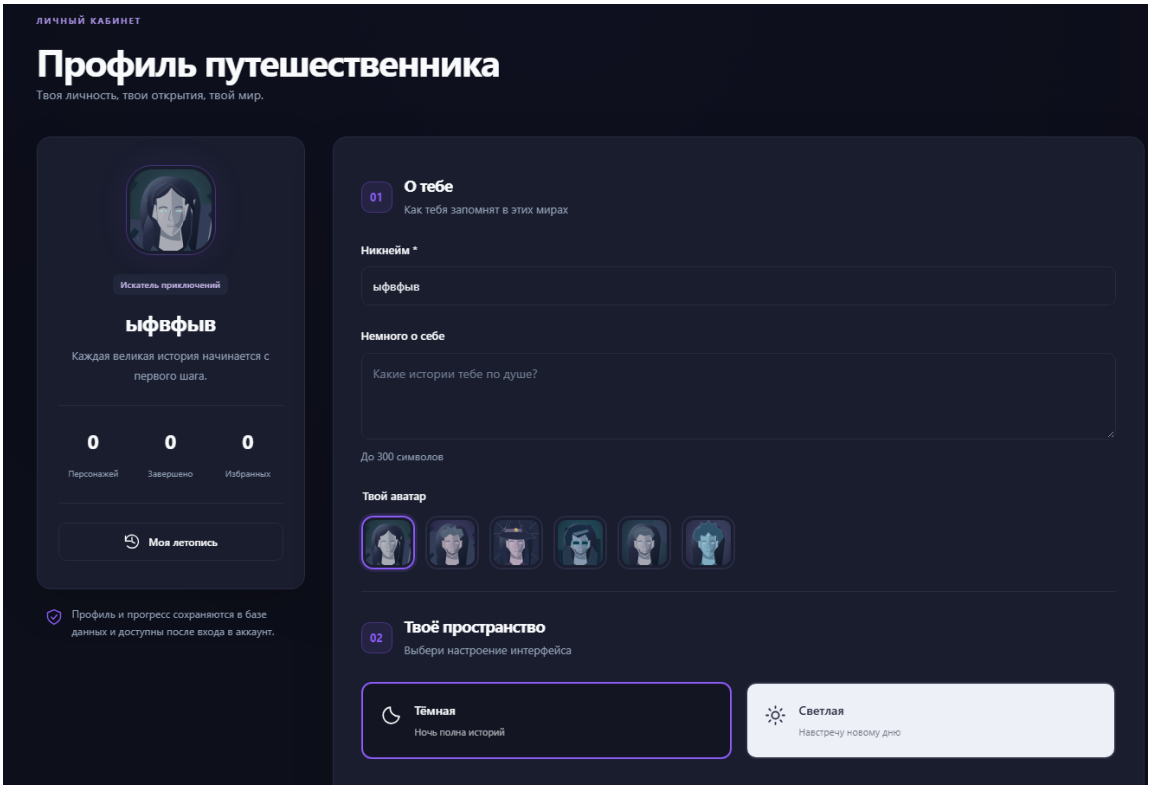


Рисунок Е.3 – Страница профиля сайта

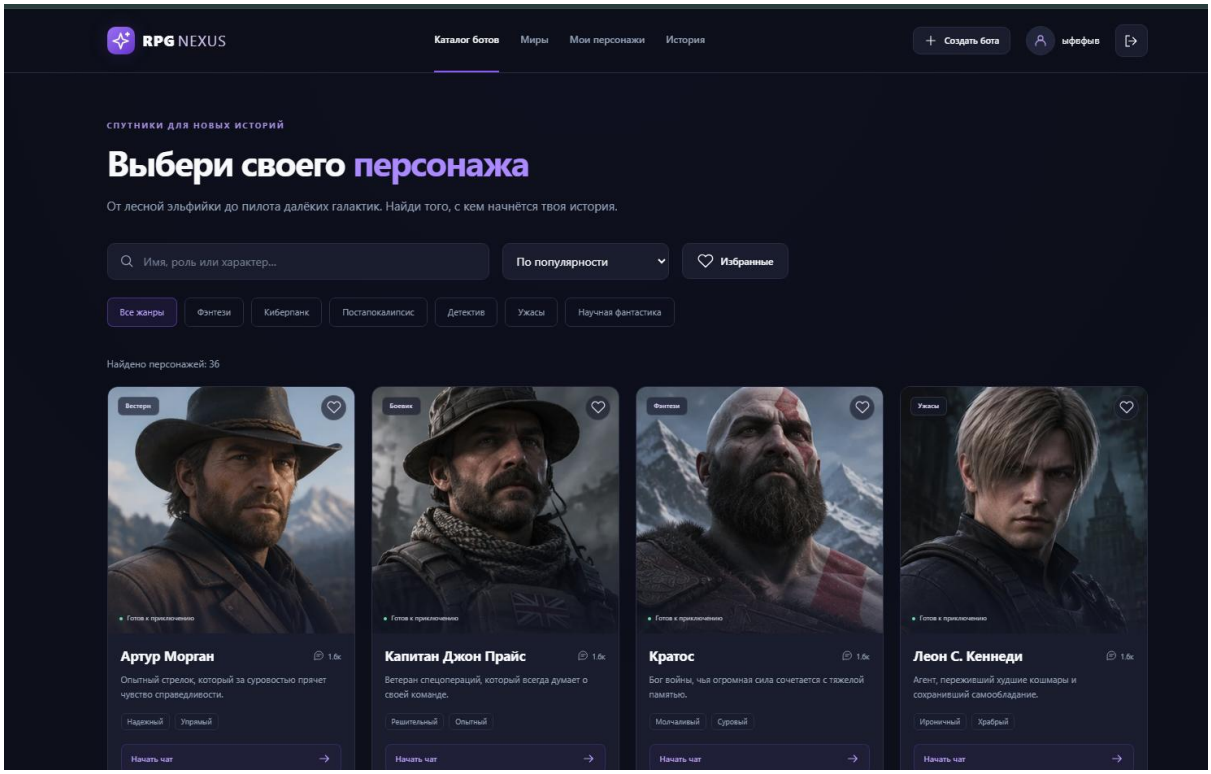


Рисунок Е.4 – Страница каталога ботов сайта

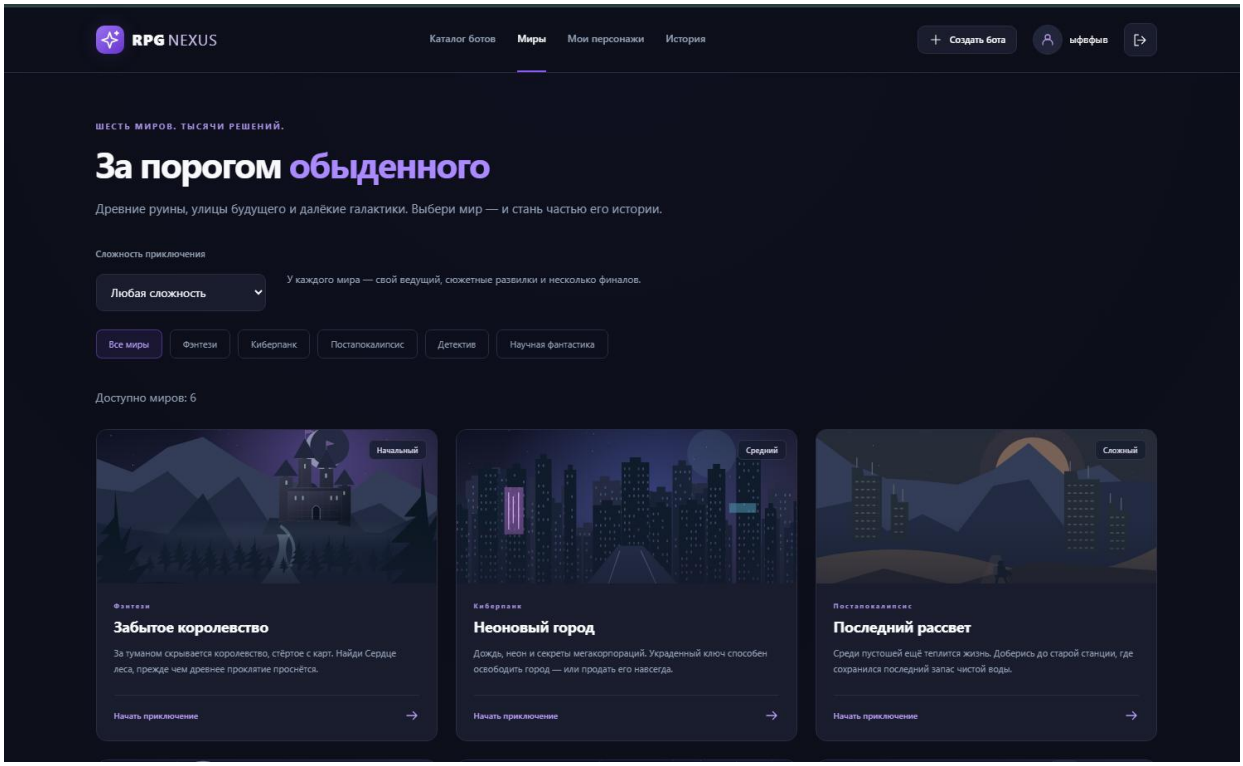


Рисунок Е.5 – Страница миров сайта

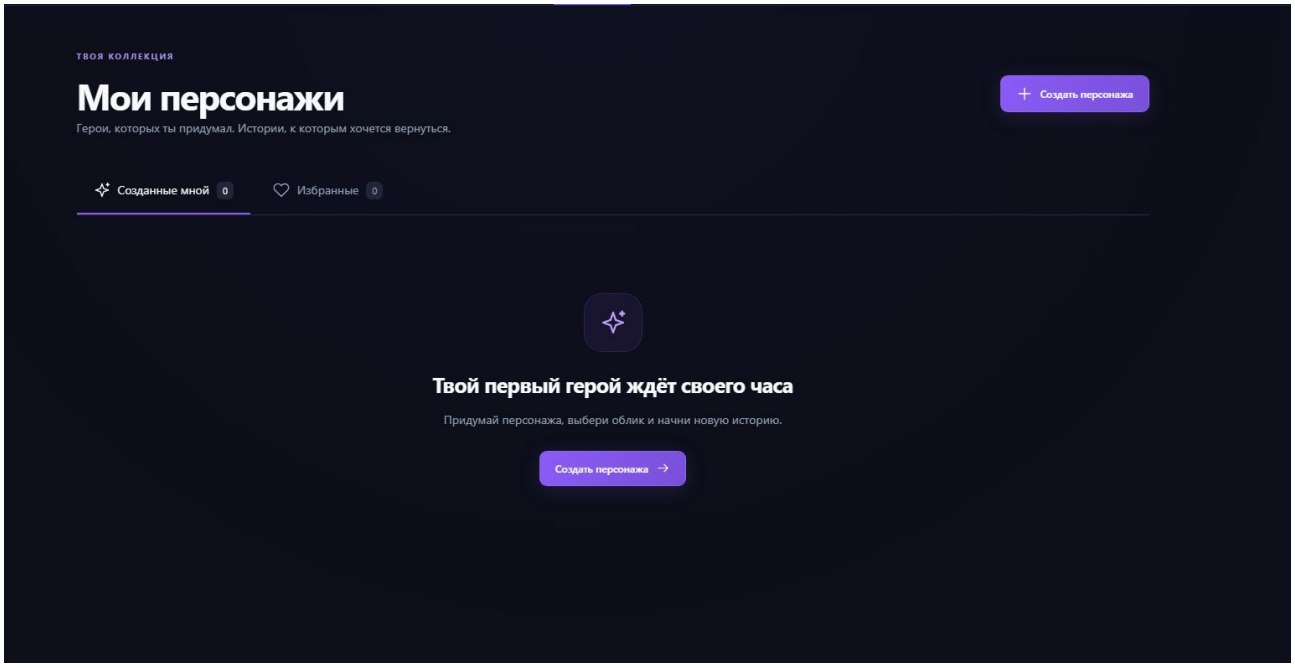


Рисунок Е.6 – Страница моих персонажей

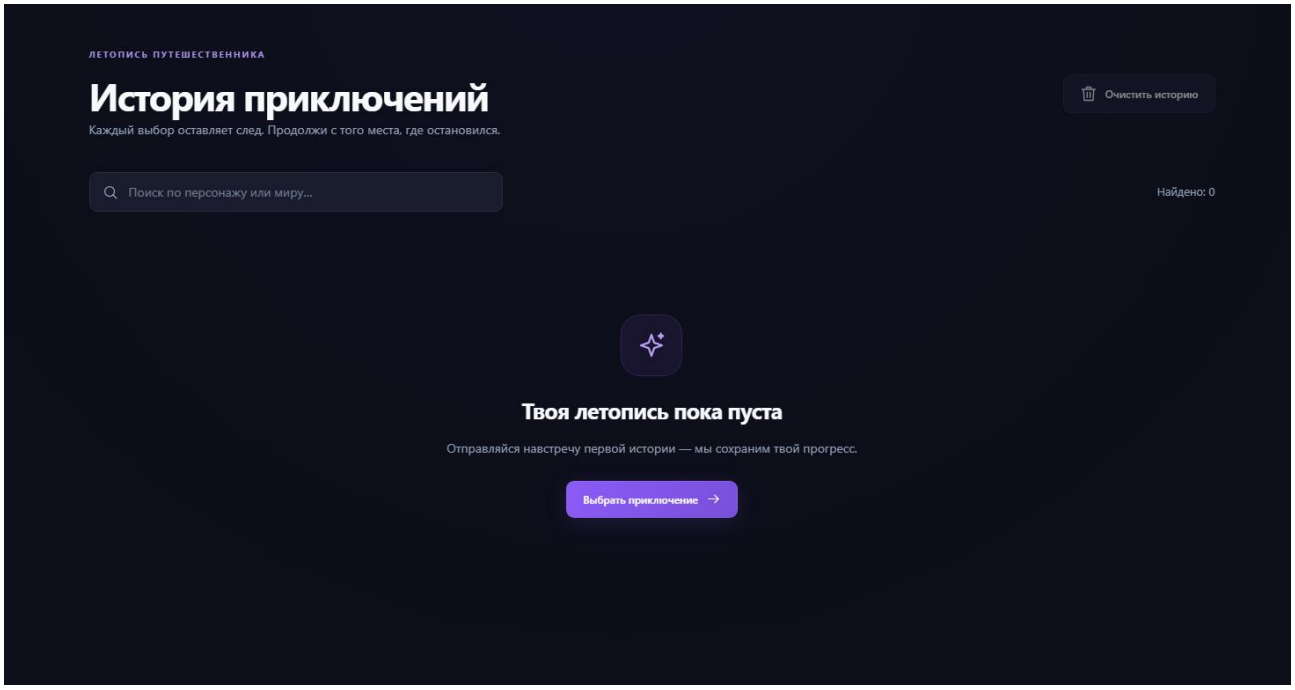


Рисунок Е.7 – Страница истории чатов

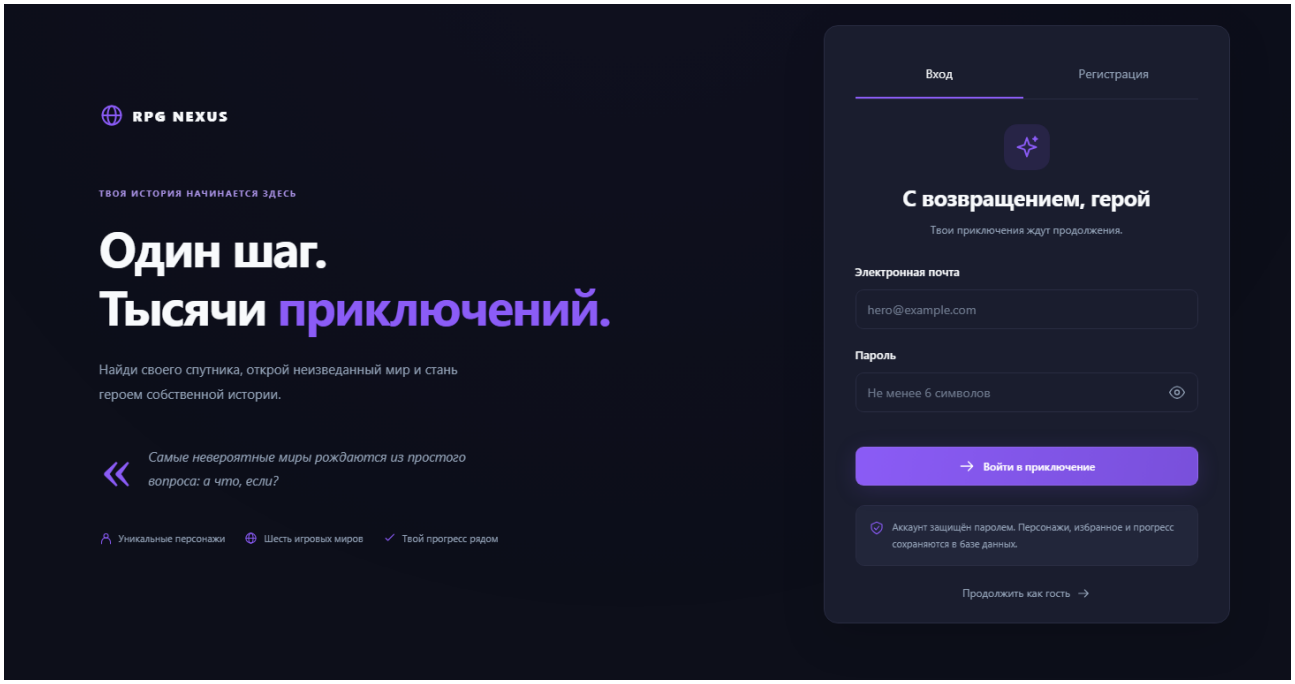


Рисунок Е.8 – Страница входа в аккаунт сайта

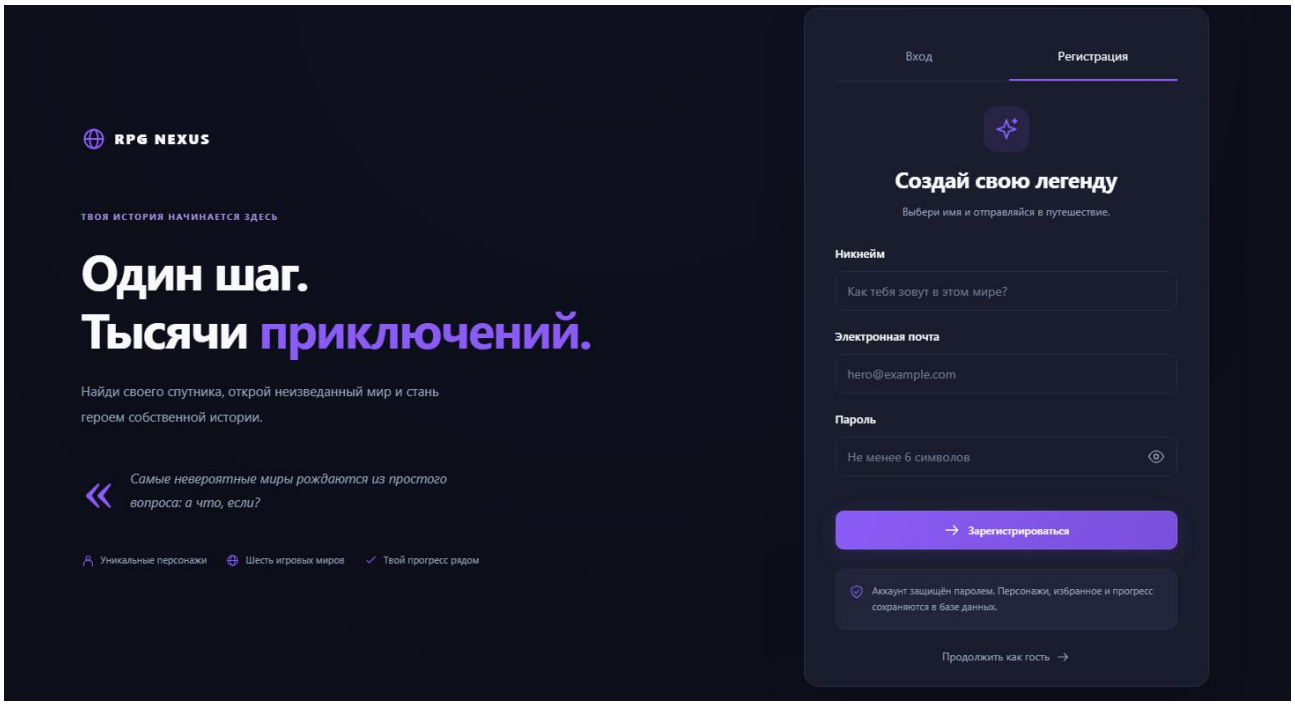


Рисунок Е.9 – Страница регистрации аккаунта сайта

Приложение Ж
(обязательное)

Листинг программы

```
CREATE TABLE users (
 id BIGSERIAL PRIMARY KEY,
 email VARCHAR(120) NOT NULL,
 password_hash TEXT NOT NULL,
 role VARCHAR(10) NOT NULL DEFAULT 'user',
 is_active BOOLEAN NOT NULL DEFAULT TRUE,
 created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE UNIQUE INDEX users_email_lower_uq ON users(lower(email));
CREATE TABLE profiles (
 user_id BIGINT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
 nickname VARCHAR(30) NOT NULL,
 avatar_url TEXT NOT NULL,
 bio VARCHAR(300) NOT NULL DEFAULT '',
 theme VARCHAR(10) NOT NULL DEFAULT 'dark'
);
```

Фрагмент схемы игровых сессий:

```
CREATE TABLE game_sessions (
 id UUID PRIMARY KEY,
 user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 character_id BIGINT NOT NULL REFERENCES characters(id),
 world_id BIGINT NOT NULL REFERENCES worlds(id),
```

```
    current_node_id BIGINT REFERENCES scenario_nodes(id),
    hero_name_snapshot VARCHAR(30) NOT NULL,
    character_name_snapshot VARCHAR(50) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'active',
    level INTEGER NOT NULL DEFAULT 1 CHECK (level >= 1),
    hp INTEGER NOT NULL DEFAULT 100 CHECK (hp >= 0),
    mp INTEGER NOT NULL DEFAULT 50 CHECK (mp >= 0),
    xp INTEGER NOT NULL DEFAULT 0 CHECK (xp >= 0),
    gold INTEGER NOT NULL DEFAULT 50 CHECK (gold >= 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
  );
```

Пример серверной обработки хода:

```
  async function handleAction(client, userId, sessionId, actionId) {
    await client.query('BEGIN');
    try {
      const r = await client.query(
        `SELECT * FROM game_sessions WHERE id=$1 AND user_id=$2 FOR UPDATE`,
        [sessionId, userId]
      );
      if (!r.rows.length) throw new Error('Сессия не найдена');
      // Проверка доступного действия, стоимости и предметов
      // Генерация D20, применение эффектов и сохранение сообщений
      await client.query('COMMIT');
    } catch (error) {
      await client.query('ROLLBACK');
      throw error;
```

```
  }
}
```

Пример проверки токена:

```
function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Требуется вход' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Недействительный токен' });
  }
}
```